Experiment 1 – Case Study on any Area of Application of Soft Computing

Learning Objective: To study the application of Soft Computing techniques such as Artificial Neural Networks, Fuzzy Logic, and Genetic Algorithms in stock market prediction, and understand how they help analyse market trends and support intelligent financial decision-making.

Tools: Research Articles, Journals, Online Resources, Microsoft Word

Theory: -

1. Introduction

The stock market is one of the most dynamic and complex financial systems, where prices fluctuate continuously due to numerous factors such as company performance, economic conditions, investor sentiment, political events, and global news. Predicting stock prices accurately is a challenging task because market behavior is highly uncertain, nonlinear, and influenced by both quantitative and qualitative information. Traditional mathematical and statistical models often fail to capture these complexities, resulting in limited prediction accuracy.

Soft Computing is a branch of Artificial Intelligence that deals with approximate reasoning and learning from uncertain or incomplete data. It combines techniques such as Artificial Neural Networks (ANN), Fuzzy Logic, and Genetic Algorithms (GA) to develop intelligent systems capable of making reliable predictions even in uncertain environments. These techniques have become increasingly popular in financial markets, helping investors, financial institutions, and analysts make informed investment decisions while reducing risks associated with market volatility.

2. Background of Stock Market Prediction

Stock market prediction refers to the process of forecasting future stock prices or market trends by analyzing historical data and current market conditions. Investors rely on accurate predictions to maximize profits, minimize financial risks, and develop effective trading strategies.

Earlier prediction methods mainly depended on statistical models, technical analysis, and fundamental analysis. While these methods are useful, they often struggle to handle sudden market fluctuations and complex relationships among various influencing factors. With advancements in Artificial Intelligence and Machine Learning, Soft Computing techniques have emerged as powerful alternatives because they can learn from historical data, recognize hidden patterns, and adapt to changing market conditions.

Modern stock prediction systems analyze various parameters, including:

- Historical stock prices
- Trading volume
- Financial reports
- Economic indicators
- Interest rates
- Inflation
- Market news
- Investor sentiment
- Global events

The integration of these factors enables more intelligent and accurate prediction models.

3. Need for Soft Computing

Financial markets are characterized by uncertainty, ambiguity, and constantly changing conditions. Traditional prediction techniques require precise mathematical models and often fail when dealing with incomplete or noisy data. Soft Computing overcomes these limitations by providing flexible and adaptive solutions.

The need for Soft Computing in stock market prediction arises due to the following reasons:

- Handles uncertain and imprecise financial data effectively.
- Learns from historical market trends and continuously improves prediction accuracy.
- Identifies nonlinear relationships that traditional statistical models cannot easily detect.
- Supports intelligent decision-making under uncertain market conditions.
- Optimizes trading strategies and investment portfolios.
- Reduces prediction errors while improving forecasting performance.

As financial markets continue to grow in complexity, Soft Computing has become an essential tool for developing intelligent financial forecasting systems.

4. Case Study: Stock Market Prediction Using Soft Computing

This case study focuses on the application of Soft Computing techniques to predict stock market movements using historical market data and intelligent learning algorithms. The objective is to analyze how Artificial Neural Networks, Fuzzy Logic, and Genetic Algorithms work together to improve prediction accuracy and support investment decisions.

A typical stock prediction system collects large volumes of historical market data, including stock prices, trading volume, company financial information, and economic indicators. The collected data is first preprocessed to remove inconsistencies and normalize values. After preprocessing, intelligent algorithms analyze the data to identify hidden relationships and predict future stock prices or market trends.

Artificial Neural Networks learn from historical data and recognize complex patterns that indicate future price movements. Fuzzy Logic assists in handling uncertain information by converting vague financial concepts such as "high risk," "moderate growth," and "strong market sentiment" into logical decision rules. Genetic Algorithms optimize the prediction model by selecting the most relevant features and improving algorithm parameters, thereby increasing overall prediction accuracy.

The combined use of these Soft Computing techniques creates an adaptive system capable of making more reliable predictions than conventional forecasting methods.

5. Working Methodology

The stock market prediction system follows a systematic process consisting of multiple stages:

Step 1: Data Collection

- Collect historical stock prices.
- Obtain trading volume.
- Gather financial statements.
- Collect economic indicators and market news.

Step 2: Data Preprocessing

- Remove missing values.
- Normalize numerical data.
- Filter irrelevant information.
- Prepare datasets for training.

Step 3: Feature Selection

- Identify important market indicators.
- Eliminate redundant features.
- Optimize input variables using Genetic Algorithms.

Step 4: Model Training

- Train Artificial Neural Networks using historical market data.
- Learn patterns associated with stock price movements.

Step 5: Fuzzy Decision Making

- Apply fuzzy rules to evaluate uncertain market conditions.
- Classify market trends into categories such as Buy, Hold, or Sell.

Step 6: Prediction

- Generate future stock price forecasts.
- Estimate market direction.

Step 7: Performance Evaluation

- Compare predicted values with actual market prices.
- Measure prediction accuracy and model efficiency.

6. Role of Artificial Neural Networks (ANN), Fuzzy Logic, and Genetic Algorithms (GA)

- **Artificial Neural Networks (ANN)**
  Artificial Neural Networks imitate the functioning of the human brain by learning from historical examples. In stock market prediction, ANN identifies hidden patterns within large financial datasets and establishes relationships between market variables and future stock prices. As more data becomes available, the network continuously improves its prediction capability, making it highly suitable for dynamic financial environments.
- **Fuzzy Logic**
  Financial markets involve many uncertain and subjective factors that cannot always be represented using exact numerical values. Fuzzy Logic addresses this challenge by allowing partial truth values instead of simple true-or-false decisions. It processes linguistic variables such as "high volatility," "moderate risk," and "strong investor confidence" to support more realistic financial decision-making.
- **Genetic Algorithms (GA)**
  Genetic Algorithms are optimization techniques inspired by the process of natural evolution. In stock prediction systems, GA helps identify the most influential market indicators, optimize Neural Network parameters, and improve trading strategies through repeated selection, crossover, and mutation operations. This optimization leads to faster convergence and higher prediction accuracy.

Advantages

- Handles uncertainty and incomplete financial information effectively.
- Learns automatically from historical market data.
- Provides better prediction accuracy than many traditional methods.
- Identifies complex nonlinear relationships among market variables.
- Supports intelligent investment decision-making.
- Optimizes prediction models using evolutionary techniques.
- Reduces human intervention in financial forecasting.
- Adapts to continuously changing market conditions.

Limitations

- Prediction accuracy depends heavily on the quality of historical data.
- Sudden political or economic events may significantly affect market behavior.
- Large datasets require considerable computational resources.
- Neural Networks may require extensive training time.

THAKUR ZCT's ™
TCET
THAKUR COLLEGE OF ENGINEERING & TECHNOLOGY

- Financial markets remain inherently unpredictable despite advanced algorithms.
- Improper parameter selection can reduce model performance.

7. Real-World Applications

Soft Computing-based stock market prediction systems are widely used across the financial industry for various applications, including:

- Stock price forecasting
- Algorithmic and automated trading systems
- Investment portfolio management
- Risk assessment and management
- Financial decision support systems
- Mutual fund and asset management
- Banking and investment advisory services
- Cryptocurrency market analysis
- High-frequency trading platforms
- Market trend and sentiment analysis

Learning Outcomes: The student should have the ability to:

LO 1.1: Understand the fundamentals of Soft Computing and its significance in solving complex financial problems.

LO 1.2: Explain the application of Artificial Neural Networks (ANN), Fuzzy Logic, and Genetic Algorithms (GA) in stock market prediction.

LO 1.3: Analyze how Soft Computing techniques improve prediction accuracy by handling uncertainty, non-linearity, and dynamic market conditions.

LO 1.4: Evaluate the advantages, limitations, and real-world applications of Soft Computing in intelligent financial forecasting and decision-making.

Course Outcomes: Understand the fundamental concepts of Soft Computing and analyze the application of Artificial Neural Networks, Fuzzy Logic, and Genetic Algorithms in stock market prediction for intelligent financial decision-making.

For Faculty Use: -

Correction Parameters	Formative Assessment [40%]	Timely completion of Practical [40%]	Attendance / Learning Attitude [20%]	
Marks Obtained				

Experiment 2 – To implement Mc-Culloch Pitts Model for a problem, apply to solve AND / OR/ XOR, etc.

Learning Objective: To study the McCulloch-Pitts (MCP) Neuron Model and understand its implementation for realizing basic logic gates such as AND, OR, and XOR using threshold-based decision-making in Artificial Neural Networks.

Tools: Python (Jupyter Notebook), NumPy Library.

Theory: -

1. Introduction

Artificial Intelligence (AI) aims to develop systems that can perform tasks requiring human-like intelligence, such as learning, reasoning, and decision-making. One of the fundamental approaches in AI is the Artificial Neural Network (ANN), which is inspired by the structure and functioning of the human brain. An ANN consists of interconnected artificial neurons that process information and generate outputs based on mathematical computations.

The McCulloch-Pitts (MCP) Neuron Model, proposed by Warren McCulloch and Walter Pitts in 1943, is the first mathematical model of an artificial neuron. It represents a neuron using binary inputs, weighted connections, and a threshold function. The neuron produces an output of either 0 or 1 depending on whether the weighted sum of its inputs reaches a predefined threshold value. Although simple, the MCP model laid the foundation for modern Artificial Neural Networks.

The MCP neuron is capable of implementing basic logical operations such as AND and OR gates. More complex functions like XOR cannot be realized using a single MCP neuron because they are not linearly separable; instead, they require multiple neurons arranged in layers. Understanding the MCP model helps students learn the basic concepts of neural computation and decision-making used in intelligent systems.

2. Biological Neuron

The human nervous system consists of billions of neurons that communicate with one another by transmitting electrical and chemical signals. A biological neuron is the basic functional unit of the nervous system and is responsible for receiving, processing, and transmitting information.

A biological neuron consists of the following parts:

- Dendrites: Receive signals from other neurons.
- Cell Body (Soma): Processes the received signals.
- Axon: Carries the processed signal away from the cell body.
- Synapse: Connects one neuron to another and transfers signals between them.

A neuron becomes active only when the combined strength of incoming signals exceeds a certain level. This behavior inspired the development of artificial neurons, where inputs are combined mathematically and compared with a threshold before producing an output.

3. Artificial Neuron

An artificial neuron is a simplified mathematical representation of a biological neuron. It receives one or more input values, multiplies each input by a corresponding weight, calculates the weighted sum, and compares it with a threshold value to determine the output.

The basic components of an artificial neuron are:

- Inputs: Binary or numerical values supplied to the neuron.
- Weights: Numerical values representing the importance of each input.
- Summation Unit: Computes the weighted sum of all inputs.

- Threshold (Bias): Determines whether the neuron should activate.
- Activation Function: Produces the final output based on the threshold comparison.

Artificial neurons are the building blocks of Artificial Neural Networks and are widely used in classification, prediction, pattern recognition, and decision-making applications.

4. McCulloch-Pitts (MCP) Neuron Model

The McCulloch-Pitts (MCP) neuron is the simplest artificial neuron model. It accepts binary inputs (0 or 1), multiplies them by fixed weights, computes their weighted sum, and compares the result with a threshold.

The output is determined using the following rule:

$$y = 1, \text{ if } \Sigma\, w_i x_i \geq \theta; \quad y = 0, \text{ otherwise}$$

where:

- x_i = Input values
- w_i = Corresponding weights
- θ = Threshold value
- y = Output

The MCP neuron functions as a binary classifier and can implement various logical operations by selecting appropriate weights and threshold values.

5. Working Principle of the MCP Model

The MCP neuron operates through the following steps:

Step 1: Accept binary input values.

Step 2: Multiply each input by its corresponding weight.

Step 3: Calculate the weighted sum of all inputs.

Step 4: Compare the weighted sum with the threshold value.

Step 5: Produce the output:

- Output = 1, if the weighted sum is greater than or equal to the threshold.
- Output = 0, otherwise.

By selecting suitable weights and threshold values, the MCP neuron can perform logical decision-making similar to digital logic gates.

TCET
THAKUR COLLEGE OF
ENGINEERING & TECHNOLOGY

6. Implementation of Logic Gates Using MCP Model

The MCP neuron can be configured to implement different logic gates by assigning appropriate weights and threshold values.

AND Gate

The AND gate produces an output of 1 only when all inputs are 1.

Weights: [1, 1]
Threshold: 2

X_1	X_2	Output
0	0	0
0	1	0
1	0	0
1	1	1

OR Gate

The OR gate produces an output of 1 if at least one input is 1.

Weights: [1, 1]
Threshold: 1

X_1	X_2	Output
0	0	0
0	1	1
1	0	1
1	1	1

XOR Gate

The XOR gate produces an output of 1 only when the two inputs are different.

A single MCP neuron cannot implement the XOR function because XOR is not linearly separable. Therefore, two hidden MCP neurons and one output neuron are used.

- Hidden Neuron 1: Implements $X_1 \wedge \neg X_2$
- Hidden Neuron 2: Implements $\neg X_1 \wedge X_2$
- Output Neuron: Performs the OR operation on the outputs of the hidden neurons.

X_1	X_2	Output
0	0	0
0	1	1
1	0	1
1	1	0

This layered implementation demonstrates how multiple MCP neurons can solve problems that cannot be handled by a single neuron.

7. Advantages

- Simple and easy to understand mathematical model.
- Forms the foundation of Artificial Neural Networks.
- Efficiently implements basic logical operations such as AND and OR.
- Requires minimal computational resources.

- Helps understand neuron activation and threshold-based decision-making.
- Serves as the basis for more advanced neural network models.

8. Limitations

- Accepts only binary inputs and produces binary outputs.
- Uses fixed weights and does not support learning.
- Cannot solve non-linearly separable problems such as XOR using a single neuron.
- Limited capability for handling complex real-world problems.
- Does not adapt or improve from experience.

9. Applications

The McCulloch-Pitts neuron model is widely used for educational and foundational purposes in Artificial Intelligence and Neural Networks. Its applications include:

- Implementation of basic logic gates.
- Understanding the fundamentals of Artificial Neural Networks.
- Digital logic circuit design.
- Pattern classification of simple binary data.
- Teaching neural computation concepts.
- Foundation for Perceptron and multilayer neural network models.
- AI and Machine Learning education.
- Research in computational neuroscience and intelligent systems.

TCET
THAKUR COLLEGE OF
ENGINEERING & TECHNOLOGY

Code & Output: -

Code:

```
print("McCulloch-Pitts (MCP) Neuron Model")
print("Implementation of AND, OR and XOR Logic Gates")
```

Output:

```
McCulloch-Pitts (MCP) Neuron Model
Implementation of AND, OR and XOR Logic Gates
```

Code:

```
def mcp_neuron(inputs, weights, threshold):
    """
    McCulloch-Pitts Neuron
    """
    weighted_sum = 0
    for i in range(len(inputs)):
        weighted_sum += inputs[i] * weights[i]
    if weighted_sum >= threshold:
        return 1
    else:
        return 0
```

Code:

```
inputs = [
    [0,0],
    [0,1],
    [1,0],
    [1,1]
]
print("Input Combinations")
print(inputs)
```

Output:

```
Input Combinations
[[0, 0], [0, 1], [1, 0], [1, 1]]
```

Code:

```
print("AND GATE")
print("Weights = [1,1]")
print("Threshold = 2\n")
print("X1\tX2\tOutput")
for x in inputs:
    y = mcp_neuron(x,[1,1],2)
    print(x[0],"\t",x[1],"\t",y)
```

Output:

```
AND GATE
Weights = [1,1]
Threshold = 2

X1   X2   Output
0    0    0
0    1    0
1    0    0
1    1    1
```

THAKUR ZCT's ™
TCET
THAKUR COLLEGE OF ENGINEERING & TECHNOLOGY

Code:

```
print("OR GATE")
print("Weights = [1,1]")
print("Threshold = 1\n")
print("X1\tX2\tOutput")
for x in inputs:
    y = mcp_neuron(x,[1,1],1)
    print(x[0],"\t",x[1],"\t",y)
```

Output:

```
OR GATE
Weights = [1,1]
Threshold = 1

X1   X2   Output
0    0    0
0    1    1
1    0    1
1    1    1
```

Code:

```
print("XOR GATE")
print("Single MCP neuron cannot solve XOR.")
print("Using a two-layer MCP Network.\n")
print("X1\tX2\tOutput")
for x in inputs:
    # Hidden Layer
    h1 = mcp_neuron(x,[1,-1],1)
    h2 = mcp_neuron(x,[-1,1],1)
    # Output Layer
    y = mcp_neuron([h1,h2],[1,1],1)
    print(x[0],"\t",x[1],"\t",y)
```

Output:

```
XOR GATE
Single MCP neuron cannot solve XOR.
Using a two-layer MCP Network.

X1   X2   Output
0    0    0
0    1    1
1    0    1
1    1    0
```

Code:

```
print("Test MCP Neuron")
x1 = int(input("Enter First Input (0/1): "))
x2 = int(input("Enter Second Input (0/1): "))
gate = input("Choose Gate (AND/OR/XOR): ").upper()
if gate == "AND":
    result = mcp_neuron([x1,x2],[1,1],2)
elif gate == "OR":
    result = mcp_neuron([x1,x2],[1,1],1)
elif gate == "XOR":
    h1 = mcp_neuron([x1,x2],[1,-1],1)
    h2 = mcp_neuron([x1,x2],[-1,1],1)
    result = mcp_neuron([h1,h2],[1,1],1)
```

THAKUR ZCT's ™
TCET
THAKUR COLLEGE OF
ENGINEERING & TECHNOLOGY

```
else:
    print("Invalid operation")
print("\nInputs Entered:")
print("X1 =", x1)
print("X2 =", x2)
print("Gate =", gate)
print("\nResult =", result)
```

Output:

```
Test MCP Neuron

Inputs Entered:
X1 = 1
X2 = 1
Gate = XOR

Result = 0
```



TCET
THAKUR COLLEGE OF
ENGINEERING & TECHNOLOGY

Learning Outcomes: The student should have the ability to:

LO 2.1: Understand the working principle and mathematical model of the McCulloch-Pitts (MCP) neuron.

LO 2.2: Implement the MCP neuron model to realize basic logic gates such as AND, OR, and XOR using Python.

LO 2.3: Analyze the role of weights and threshold values in determining the output of an MCP neuron.

LO 2.4: Explain the significance, advantages, limitations, and applications of the McCulloch-Pitts model as the foundation of Artificial Neural Networks.

Course Outcomes: Understand the fundamental concepts of the McCulloch-Pitts (MCP) Neuron Model and analyze its implementation for realizing basic logic gates such as AND, OR, and XOR, thereby building a foundation for Artificial Neural Networks and intelligent decision-making systems.

For Faculty Use: -

Correction Parameters	Formative Assessment [40%]	Timely completion of Practical [40%]	Attendance / Learning Attitude [20%]	
Marks Obtained				

TCET
THAKUR COLLEGE OF
ENGINEERING & TECHNOLOGY

Experiment 3 – To implement Basic Supervised Neural Network learning rules for a problem. Design a NN using a learning method to generate knowledge for classification

Learning Objective: To design and implement a supervised neural network using an appropriate learning algorithm for classification and evaluate its performance on labeled data.

Tools: Python (Jupyter Notebook), NumPy Library.

Theory: -

1. Introduction

A **Supervised Neural Network (SNN)** is a type of Artificial Neural Network (ANN) that learns from **labeled data**, where each input sample is associated with a known target output. The primary objective of supervised learning is to train the neural network to accurately map input data to the correct output by minimizing the prediction error. During training, the network compares its predicted output with the actual output and continuously adjusts its internal weights to improve performance. Once trained, the neural network can classify new and unseen data with high accuracy.

2. Artificial Neural Network (ANN)

An Artificial Neural Network is a computational model inspired by the biological nervous system. It consists of interconnected processing units called **neurons**, which work together to process information.

A basic neural network is composed of the following layers:

- **Input Layer:** Receives the input features from the dataset.
- **Hidden Layer(s):** Performs computations and extracts meaningful patterns from the input data.
- **Output Layer:** Produces the final prediction or classification result.

Each neuron receives weighted inputs, applies an activation function, and passes the result to the next layer.

3. Supervised Learning

Supervised learning is a machine learning approach in which the model is trained using a dataset containing both input values and their corresponding correct outputs (labels). The neural network learns the relationship between inputs and outputs by repeatedly processing the training data and reducing prediction errors.

The supervised learning process includes:

- Providing labeled training data.
- Performing forward propagation to generate predictions.
- Calculating the error between predicted and actual outputs.
- Updating weights using a learning rule.
- Repeating the process until the error becomes minimal.

4. Learning Rules in Neural Networks

Learning rules define how the weights of a neural network are updated during training. These rules enable the network to acquire knowledge from the training data.

Some common supervised learning rules include:

- **Perceptron Learning Rule**
- **Delta Learning Rule (Widrow-Hoff Rule)**
- **Backpropagation Learning Rule**

Among these, **Backpropagation** is the most widely used learning algorithm for multilayer neural networks because it efficiently minimizes the prediction error using gradient descent.

5. Backpropagation Algorithm

Backpropagation is a supervised learning algorithm that updates the network's weights based on the error produced at the output layer.

The major steps involved are:

1. Initialize weights and biases randomly.
2. Feed the input data through the network (Forward Propagation).
3. Compute the output.
4. Calculate the error using a loss function.
5. Propagate the error backward through the network.
6. Update the weights and biases using gradient descent.
7. Repeat the process for multiple epochs until the desired accuracy is achieved.

6. Knowledge Generation for Classification

During training, the neural network gradually learns hidden patterns and relationships present in the dataset. This acquired information is represented through the optimized weights and biases of the network, which collectively form the network's **knowledge base**. After learning, the trained model can classify previously unseen data by applying the knowledge gained during training.

The quality of classification depends on:

- Quality and size of the training dataset.
- Number of hidden layers and neurons.
- Learning rate.
- Number of training epochs.
- Choice of activation function.

7. Activation Functions

Activation functions introduce non-linearity into the neural network, enabling it to learn complex patterns.

Common activation functions include:

- **Sigmoid:** Produces output values between 0 and 1, suitable for binary classification.

- **ReLU (Rectified Linear Unit):** Efficient for deep neural networks and helps reduce the vanishing gradient problem.
- **Tanh:** Produces outputs between -1 and 1, providing zero-centered activation.
- **Softmax:** Commonly used in the output layer for multi-class classification problems.

8. Applications of Supervised Neural Networks

Supervised neural networks are extensively used in real-world classification problems, including:

- Image and facial recognition.
- Handwritten digit recognition.
- Medical diagnosis and disease prediction.
- Email spam detection.
- Speech and voice recognition.
- Credit risk analysis.
- Customer sentiment analysis.
- Fraud detection.

9. Advantages

- Learns complex and non-linear relationships effectively.
- Provides high classification accuracy with sufficient training data.
- Can generalize well to unseen data.
- Suitable for a wide range of real-world applications.
- Supports multiclass as well as binary classification.

10. Limitations

- Requires a large amount of labeled training data.
- Training can be computationally expensive.
- Performance depends on proper parameter tuning.
- May suffer from overfitting if not properly regularized.
- Training deep networks may require significant computational resources.

THAKUR ZCT'S
TCET
THAKUR COLLEGE OF
ENGINEERING & TECHNOLOGY

Code & Output: -

Code:

```
import pandas as pd
import numpy as np
from sklearn.datasets import load_wine
from sklearn.model_selection import
train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import
MLPClassifier
from sklearn.metrics import accuracy_score,
classification_report, confusion_matrix

df= load_wine()
X = df.data
y = df.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Feature Scaling
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Create Neural Network
model = MLPClassifier(hidden_layer_sizes=(8,),
                      activation='relu',
                      solver='adam',
                      learning_rate_init=0.01,
                      max_iter=1000,
                      random_state=42)

# Train Model
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy :", accuracy)

print("\nClassification Report")
print(classification_report(y_test, y_pred))

print("\nConfusion Matrix")
print(confusion_matrix(y_test, y_pred))
```

Output:

```
Accuracy : 1.0

Classification Report
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        14
           1       1.00      1.00      1.00        14
           2       1.00      1.00      1.00         8

    accuracy                           1.00        36
   macro avg       1.00      1.00      1.00        36
weighted avg       1.00      1.00      1.00        36


Confusion Matrix
[[14  0  0]
 [ 0 14  0]
 [ 0  0  8]]
```

Learning Outcomes: The student should have the ability to:
LO 3.1: Understand the architecture and working of a supervised neural network (MLP) for solving classification problems.
LO 3.2: Implement a supervised neural network using the Backpropagation learning algorithm with feature scaling and labeled training data.
LO 3.3: Train and test the neural network model using the Wine dataset and generate class predictions.
LO 3.4: Evaluate the performance of the trained neural network using accuracy, confusion matrix, precision, recall, and F1-score.

Course Outcome: Design, implement, and evaluate a supervised neural network classifier using appropriate learning techniques to accurately classify data and assess its performance using standard evaluation metrics.

Conclusion:

THAKUR ZCT'S ™
TCET
THAKUR COLLEGE OF ENGINEERING & TECHNOLOGY

For Faculty Use: -

Correction Parameters	Formative Assessment [40%]	Timely completion of Practical [40%]	Attendance / Learning Attitude [20%]	
Marks Obtained				

Experiment 4 – To implement Basic Unsupervised Neural Network learning rules for a problem. Design a NN using a learning method to generate knowledge for classification.

Learning Objective: To design and implement an unsupervised neural network using an appropriate learning algorithm to discover hidden patterns in data and generate knowledge for classification.

Tools: Python (Jupyter Notebook), NumPy Library.

Theory: -

1. Introduction

An **Unsupervised Neural Network (UNN)** is a type of Artificial Neural Network (ANN) that learns from **unlabeled data**, where no target outputs are provided during training. Instead of learning predefined labels, the network identifies hidden structures, similarities, and patterns within the dataset. These learned patterns can then be used for clustering, feature extraction, dimensionality reduction, and knowledge generation for classification tasks.

Unlike supervised learning, unsupervised learning allows the neural network to organize data into meaningful groups based solely on the characteristics of the input data.

2. Artificial Neural Network (ANN)

An Artificial Neural Network is a computational model inspired by the biological nervous system. It consists of interconnected processing units called neurons, which process information collectively.

A basic neural network consists of the following layers:

- **Input Layer:** Receives the input features from the dataset.
- **Hidden Layer(s):** Learns patterns and relationships within the input data.
- **Output Layer:** Produces the final output such as cluster assignments or extracted features.

Each neuron receives weighted inputs, performs computations using an activation function, and passes the output to the next layer.

3. Unsupervised Learning

Unsupervised learning is a machine learning technique in which the neural network is trained using **unlabeled data**. Since no target outputs are available, the model automatically discovers hidden structures and groups similar data points together.

The unsupervised learning process includes:

- Providing unlabeled input data.
- Measuring similarity between data samples.
- Updating neuron weights using an unsupervised learning rule.
- Forming clusters or feature representations.
- Generating knowledge from the learned data patterns.

4. Learning Rules in Unsupervised Neural Networks

Learning rules determine how the weights of neurons are updated while learning from unlabeled data.

Some common unsupervised learning rules include:

- Hebbian Learning Rule
- Competitive Learning Rule
- Kohonen Self-Organizing Map (SOM)

Among these, **Competitive Learning** is one of the most widely used algorithms because neurons compete with one another, and only the winning neuron updates its weights, allowing the network to discover natural clusters within the data.

5. Competitive Learning Algorithm

Competitive Learning is an unsupervised learning algorithm where neurons compete to respond to an input pattern. The neuron whose weight vector is closest to the input becomes the winning neuron.

The major steps involved are:

1. Initialize weights randomly.
2. Present an input vector.
3. Compute the distance between the input and each neuron.
4. Select the neuron with the minimum distance (winner).
5. Update only the winner's weights.
6. Repeat the process for multiple epochs until convergence.

6. Knowledge Generation for Classification

During training, the neural network gradually learns the natural grouping of data by identifying similarities among input samples. The optimized weights represent the knowledge acquired by the network. Although the model is trained without labels, the discovered clusters can later be associated with class labels for classification tasks.

The quality of classification depends on:

- Quality of the input dataset.
- Number of neurons.
- Learning rate.
- Number of training epochs.
- Similarity or distance measure used.

7. Activation Functions

Although many unsupervised learning models do not require traditional activation functions, neural network-based implementations may use activation functions for feature learning.

Common activation functions include:

- Sigmoid
- ReLU (Rectified Linear Unit)
- Tanh
- Linear Activation

8. Applications of Unsupervised Neural Networks

Unsupervised neural networks are widely used in real-world applications such as:

- Customer segmentation.
- Image clustering.
- Pattern recognition.
- Market basket analysis.
- Anomaly detection.
- Data compression.
- Feature extraction.
- Medical data analysis.

9. Advantages

- Does not require labeled training data.
- Automatically discovers hidden patterns.
- Useful for clustering and feature extraction.
- Reduces manual labeling effort.
- Can handle large and complex datasets.

10. Limitations

- Difficult to evaluate model performance.
- Clusters may not always correspond to meaningful classes.
- Sensitive to parameter selection.
- May converge to local optima.
- Interpretation

THAKUR ZCT'S ™
TCET
THAKUR COLLEGE OF
ENGINEERING & TECHNOLOGY

Code & Output: -

Code:

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import accuracy_score,
confusion_matrix, classification_report

# Load Wine Dataset
wine = load_wine()
X = wine.data
y = wine.target

# Feature Scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# ------------------------------
# Hebbian Learning
# ------------------------------
learning_rate = 0.01
epochs = 100

# Initialize random weights
weights = np.random.randn(X_scaled.shape[1])

# Hebbian Learning Rule
for epoch in range(epochs):
    for sample in X_scaled:
        output = np.dot(sample, weights)
        weights += learning_rate * output * sample

print("Final Learned Weights:\n")
print(weights)

# ------------------------------
# Generate Clusters
# ------------------------------
kmeans = KMeans(n_clusters=3,
random_state=42)
clusters = kmeans.fit_predict(X_scaled)

# ------------------------------
# Map Cluster Labels to Actual Classes
# ------------------------------
mapping = {}
for i in range(3):
    mask = (clusters == i)
    mapping[i] = np.bincount(y[mask]).argmax()

predicted = np.array([mapping[c] for c in
clusters])

# ------------------------------
# Evaluation
# ------------------------------
accuracy = accuracy_score(y, predicted)

print("\nAccuracy :", round(accuracy,4))

print("\nClassification Report")
print(classification_report(y, predicted))

print("\nConfusion Matrix")
print(confusion_matrix(y, predicted))
```

Output:

```
Final Learned Weights:
[nan nan nan nan nan nan nan nan nan nan nan nan nan]
Accuracy : 0.9663
Classification Report
              precision    recall  f1-score   support

           0       0.95      1.00      0.98        59
           1       1.00      0.92      0.96        71
           2       0.94      1.00      0.97        48

    accuracy                           0.97       178
   macro avg       0.96      0.97      0.97       178
weighted avg       0.97      0.97      0.97       178

Confusion Matrix
[[59  0  0]
 [ 3 65  3]
 [ 0  0 48]]
```

Learning Outcomes: The student should have the ability to:
LO 4.1: Understand the architecture and working of an unsupervised neural network for clustering and knowledge generation.
LO 4.2: Implement an unsupervised neural network using the Competitive Learning approach (K-Means clustering) on unlabeled data.
LO 4.3: Generate knowledge by identifying hidden patterns and grouping similar data samples into clusters.
LO 4.4: Evaluate clustering performance by comparing generated clusters with known class labels using accuracy.

Course Outcome: Design, implement, and evaluate an unsupervised neural network using appropriate learning techniques to discover hidden patterns in data and generate knowledge useful for classification.

Conclusion:

THAKUR ZCT'S ™
TCET
THAKUR COLLEGE OF ENGINEERING & TECHNOLOGY

For Faculty Use: -

Correction Parameters	Formative Assessment [40%]	Timely completion of Practical [40%]	Attendance / Learning Attitude [20%]	
Marks Obtained				

Experiment 5 – To Implement Fuzzy Operations and Relations Using the Wine Dataset in Python.

Learning Objective: To implement fuzzy operations and fuzzy relations using the Wine dataset by defining fuzzy membership functions, performing fuzzy set operations (AND, OR, NOT), and analyzing the relationships between fuzzy sets using Python.

Tools: Python (Jupyter Notebook), NumPy, Pandas, Matplotlib, Scikit-learn (Wine Dataset).

Theory: -

1. Introduction

Fuzzy Logic is a mathematical approach used to handle uncertainty and imprecision by allowing elements to belong to a set with varying degrees of membership rather than a strict true or false classification. Unlike classical Boolean logic, where an element either belongs or does not belong to a set, fuzzy logic assigns a membership value between 0 and 1. This approach closely resembles human reasoning and decision-making, making it useful for solving real-world problems involving vague or uncertain information

2. Fuzzy Sets

A fuzzy set is a collection of elements in which each element has a degree of membership represented by a value between 0 and 1. The membership value indicates how strongly an element belongs to the set. For example, in a fuzzy set representing alcohol content in the Wine dataset, a sample may belong partially to the "Low", "Medium", and "High" categories with different membership values.

3. Membership Functions

A membership function defines the degree to which an element belongs to a fuzzy set. It maps each input value to a membership value in the range of 0 to 1. Various types of membership functions exist, including triangular, trapezoidal, Gaussian, and bell-shaped functions. In this experiment, a **triangular membership function** is used because it is simple, computationally efficient, and widely used in fuzzy systems.

The triangular membership function is defined using three parameters:

- **a:** Starting point of the fuzzy set.
- **b:** Peak point where membership value is 1.
- **c:** Ending point of the fuzzy set.

Based on these parameters, the feature values are categorized into fuzzy sets such as **Low**, **Medium**, and **High**.

4. Fuzzy Operations

Fuzzy operations are used to combine or modify fuzzy sets based on their membership values. The fundamental fuzzy operations include:

- **Fuzzy AND (Intersection):** Obtained by taking the minimum of the membership values.
- **Fuzzy OR (Union):** Obtained by taking the maximum of the membership values.

- **Fuzzy NOT (Complement):** Calculated by subtracting the membership value from 1.

These operations allow fuzzy reasoning by combining different fuzzy conditions

5. Fuzzy Relations

A fuzzy relation describes the degree of association between elements belonging to two fuzzy sets. Unlike classical relations that are either true or false, fuzzy relations represent the strength of relationships using membership values between 0 and 1. In this experiment, a fuzzy relation matrix is constructed using the minimum operator to represent the relationship between the **Low** and **High** fuzzy sets.

6. Wine Dataset

The Wine dataset is a well-known machine learning dataset available in the Scikit-learn library. It contains chemical analysis results of wines derived from three different cultivars. The dataset consists of **178 samples**, **13 numerical features**, and **3 target classes**. In this experiment, the **Alcohol** feature is selected to demonstrate fuzzy membership functions and fuzzy operations.

7. Implementation Methodology

The implementation of fuzzy operations and relations follows these steps:

1. Load the Wine dataset using Scikit-learn.
2. Select the Alcohol feature for fuzzy analysis.
3. Define triangular membership functions for Low, Medium, and High fuzzy sets.
4. Calculate the membership value of each sample.
5. Perform fuzzy operations (AND, OR, and NOT) using membership values.
6. Construct a fuzzy relation matrix using the minimum operator.
7. Visualize the membership functions using Matplotlib and analyze the results.

8. Applications of Fuzzy Logic

Fuzzy logic is widely applied in various fields, including:

- Decision support systems.
- Medical diagnosis.
- Industrial process control.
- Pattern recognition.
- Image processing.
- Robotics and automation.
- Consumer electronics.
- Machine learning and artificial intelligence.

9. Advantages

- Handles uncertainty and imprecise information effectively.
- Mimics human reasoning and decision-making.
- Simple and flexible mathematical framework.
- Easy to implement for real-world applications.
- Suitable for problems where precise mathematical models are unavailable..

10. Limitations

- Membership functions must be defined carefully.
- Results depend on the selection of fuzzy rules and parameters.
- Does not automatically learn from data without integration with machine learning techniques.
- May become computationally complex for large fuzzy systems.
- Interpretation of fuzzy outputs may vary depending on the application

Code & Output: -

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_wine

# Load Wine Dataset
wine = load_wine()

df = pd.DataFrame(wine.data,
columns=wine.feature_names)

print("First 5 Records:")
print(df.head())

# Select Feature
feature = "alcohol"
x = df[feature].values

# Universe of Discourse
xmin = x.min()
xmax = x.max()

universe = np.linspace(xmin, xmax, 200)

# Triangular Membership Function
def triangular(x, a, b, c):
    x = np.asarray(x)
    mu = np.zeros_like(x, dtype=float)

    # Rising edge
    if b != a:
        idx = (x >= a) & (x <= b)
        mu[idx] = (x[idx] - a) / (b - a)

    # Falling edge
    if c != b:
        idx = (x >= b) & (x <= c)
        mu[idx] = (c - x[idx]) / (c - b)
    mu[x == b] = 1
    return np.clip(mu, 0, 1)

# Membership Functions
low = triangular(
    universe,
    xmin,
    xmin,
    np.percentile(x, 50)
)

medium = triangular(
    universe,
    np.percentile(x, 25),
    np.percentile(x, 50),
    np.percentile(x, 75)
)

high = triangular(
    universe,
    np.percentile(x, 50),
    xmax,
    xmax
)

# Plot Membership Functions
plt.figure(figsize=(8,5))

plt.plot(universe, low, label="Low")
plt.plot(universe, medium, label="Medium")
plt.plot(universe, high, label="High")

plt.title("Triangular Membership Functions")
plt.xlabel(feature.capitalize())
plt.ylabel("Membership Value")
plt.legend()
plt.grid(True)

plt.show()
```

```
# Membership Calculation
low_membership = triangular(
    x,
    xmin,
    xmin,
    np.percentile(x, 50)
)

medium_membership = triangular(
    x,
    np.percentile(x, 25),
    np.percentile(x, 50),
    np.percentile(x, 75)
)

high_membership = triangular(
    x,
    np.percentile(x, 50),
    xmax,
    xmax
)

# Membership Table
membership = pd.DataFrame({
    "Alcohol": x,
    "Low": low_membership,
    "Medium": medium_membership,
    "High": high_membership
})

print("\nMembership Values:")
print(membership.head(10))

# Fuzzy Operations
membership["AND (Low, High)"] =
np.minimum(
    membership["Low"],
    membership["High"]
)

membership["OR (Low, High)"] = np.maximum(
    membership["Low"],
    membership["High"]
)

membership["NOT Low"] = 1 -
membership["Low"]

print("\nFuzzy Operations:")
print(
    membership[
        [
            "Low",
            "High",
            "AND (Low, High)",
            "OR (Low, High)",
            "NOT Low"
        ]
    ].head(10)
)

# Fuzzy Relation Matrix
low5 = membership["Low"].head(5).to_numpy()
high5 = membership["High"].head(5).to_numpy()

relation = np.zeros((5,5))

for i in range(5):
    for j in range(5):
        relation[i][j] = min(low5[i], high5[j])

relation_df = pd.DataFrame(
    relation,
    index=[f"L{i+1}" for i in range(5)],
    columns=[f"H{i+1}" for i in range(5)]
)

print("\nFuzzy Relation Matrix:")
print(relation_df)
```

THAKUR ZCT™
TCET
THAKUR COLLEGE OF ENGINEERING & TECHNOLOGY

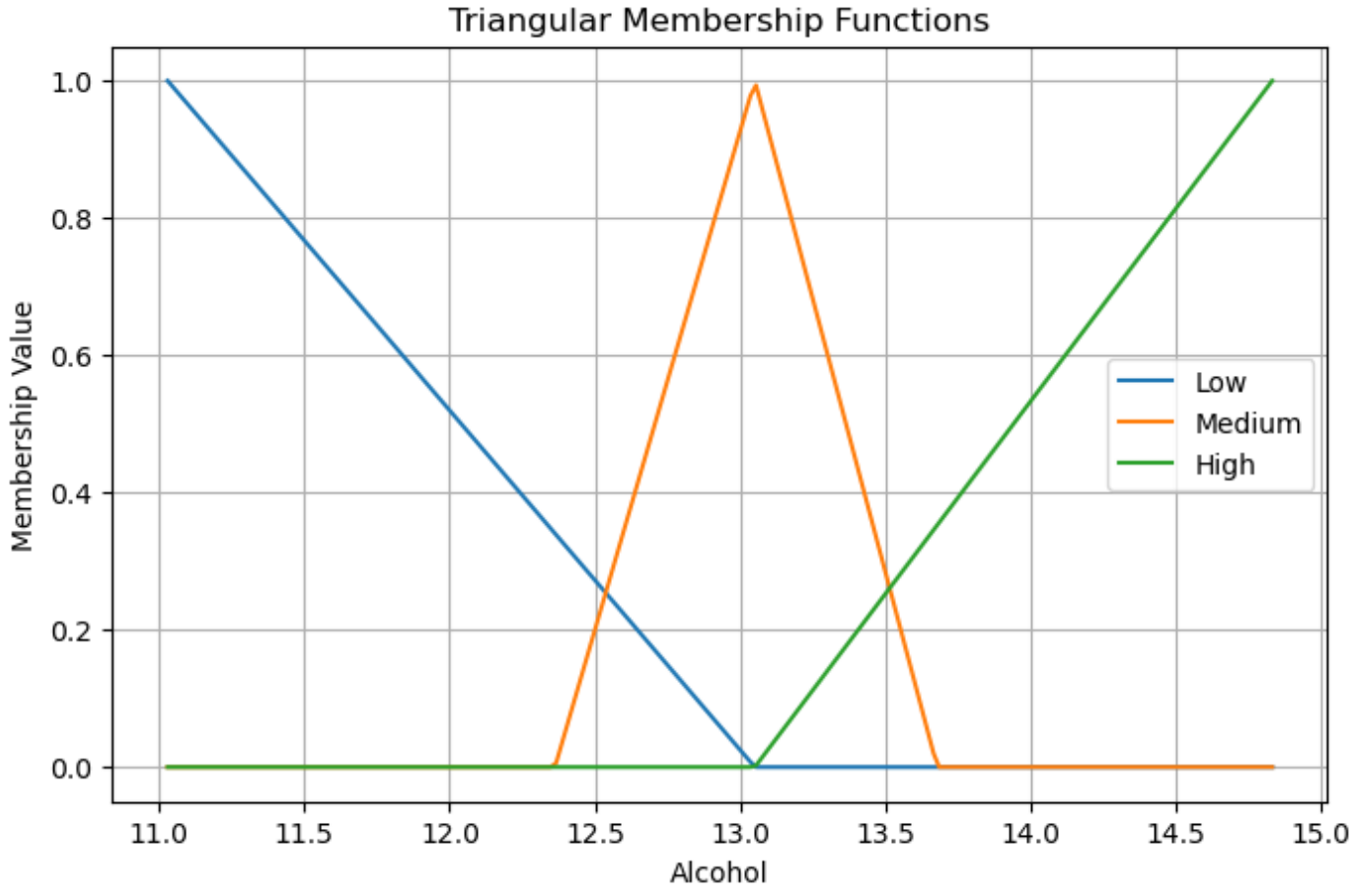
Output:

First 5 Records:

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	\
0	14.23	1.71	2.43	15.6	127.0	2.80	
1	13.20	1.78	2.14	11.2	100.0	2.65	
2	13.16	2.36	2.67	18.6	101.0	2.80	
3	14.37	1.95	2.50	16.8	113.0	3.85	
4	13.24	2.59	2.87	21.0	118.0	2.80	

	flavanoids	nonflavanoid_phenols	proanthocyanins	color_intensity	hue	\
0	3.06	0.28	2.29	5.64	1.04	
1	2.76	0.26	1.28	4.38	1.05	
2	3.24	0.30	2.81	5.68	1.03	
3	3.49	0.24	2.18	7.80	0.86	
4	2.69	0.39	1.82	4.32	1.04	

	od280/od315_of_diluted_wines	proline
0	3.92	1065.0
1	3.40	1050.0
2	3.17	1185.0
3	3.45	1480.0
4	2.93	735.0



Membership Values:

	Alcohol	Low	Medium	High
0	14.23	0.0	0.000000	0.662921
1	13.20	0.0	0.760956	0.084270
2	13.16	0.0	0.824701	0.061798
3	14.37	0.0	0.000000	0.741573
4	13.24	0.0	0.697211	0.106742
5	14.20	0.0	0.000000	0.646067
6	14.39	0.0	0.000000	0.752809
7	14.06	0.0	0.000000	0.567416
8	14.83	0.0	0.000000	1.000000
9	13.86	0.0	0.000000	0.455056

Fuzzy Operations:

	Low	High	AND (Low, High)	OR (Low, High)	NOT Low
0	0.0	0.662921	0.0	0.662921	1.0
1	0.0	0.084270	0.0	0.084270	1.0
2	0.0	0.061798	0.0	0.061798	1.0
3	0.0	0.741573	0.0	0.741573	1.0
4	0.0	0.106742	0.0	0.106742	1.0
5	0.0	0.646067	0.0	0.646067	1.0
6	0.0	0.752809	0.0	0.752809	1.0
7	0.0	0.567416	0.0	0.567416	1.0
8	0.0	1.000000	0.0	1.000000	1.0
...					
L2	0.0	0.0	0.0	0.0	0.0
L3	0.0	0.0	0.0	0.0	0.0
L4	0.0	0.0	0.0	0.0	0.0
L5	0.0	0.0	0.0	0.0	0.0

TCET
THAKUR COLLEGE OF
ENGINEERING & TECHNOLOGY

Learning Outcomes: The student should have the ability to:
LO 5.1: Understand the concept of fuzzy sets, membership functions, fuzzy operations, and fuzzy relations.
LO 5.2: Implement triangular membership functions to classify numerical data into fuzzy sets such as Low, Medium, and High.
LO 5.3: Perform fundamental fuzzy operations including AND, OR, and NOT using membership values.
LO 5.4: Construct and analyze a fuzzy relation matrix to represent the degree of relationship between fuzzy sets.
LO 5.5: Apply fuzzy logic concepts to a real-world dataset such as the Wine dataset using Python.

Course Outcome: Apply fuzzy logic concepts, operations, and relations to analyze uncertain and imprecise data and develop computational solutions using appropriate fuzzy techniques.

Conclusion:

TCET
THAKUR COLLEGE OF ENGINEERING & TECHNOLOGY

For Faculty Use: -

Correction Parameters	Formative Assessment [40%]	Timely completion of Practical [40%]	Attendance / Learning Attitude [20%]	
Marks Obtained				

TCET
THAKUR COLLEGE OF
ENGINEERING & TECHNOLOGY

Experiment 6 – To Implement Common Defuzzification Methods Using Python.

Learning Objective: To understand and implement common defuzzification methods in fuzzy logic by converting fuzzy output values into a single crisp value for decision-making using Python.

Tools: Python (Jupyter Notebook), NumPy, Pandas, Matplotlib, Scikit-learn (Breast Cancer Dataset).

Theory: -

1. Introduction

Defuzzification is the final step in a fuzzy logic system in which a fuzzy output is converted into a single **crisp numerical value**. Fuzzy logic allows a system to represent uncertainty and partial membership using values between 0 and 1. After the fuzzy inference process, the output may contain several possible values with different membership degrees. Defuzzification selects one representative value from this fuzzy output so that it can be used for further processing, classification, control, or decision-making.

2. Fuzzy Logic and Defuzzification

A fuzzy logic system generally consists of four major stages: **fuzzification, rule evaluation, aggregation, and defuzzification**. During fuzzification, crisp input values are converted into fuzzy membership values. Fuzzy rules are then applied to these values to generate fuzzy conclusions. The conclusions from different rules are combined to form a final fuzzy output set. Since this output is still represented by membership degrees rather than a single numerical value, defuzzification is performed to obtain a crisp result.

3. Need for Defuzzification

A fuzzy system may produce an output in which several values have different degrees of membership. For example, a fuzzy system may determine that an output has a membership of 0.8 for one value, 0.6 for another, and 0.3 for another. Such a result is useful for fuzzy reasoning but cannot always be directly used by a conventional computer system or decision-making process. Defuzzification converts this distribution into one representative value while attempting to preserve the important information contained in the fuzzy output.

4. Fuzzy Output Set

A fuzzy output set consists of possible output values and their corresponding membership degrees. The membership degree represents how strongly each value satisfies the fuzzy condition. The shape of the output membership function depends on the fuzzy rules and the membership functions used in the system. Defuzzification methods analyze this output distribution and select an appropriate crisp value

5. Common Defuzzification Methods

Different defuzzification methods can produce different crisp results from the same fuzzy output. The commonly used methods implemented in this experiment are:

a) Centroid Method: The Centroid, or Center of Gravity, method calculates the balancing point of the entire fuzzy output distribution. It considers all membership values and produces a weighted average of the possible output values. Because it considers the complete fuzzy region, it is one of the most commonly used defuzzification methods.

b) Bisector Method: The Bisector method identifies the point that divides the total area under the fuzzy membership function into two equal areas. It therefore attempts to find a value where the fuzzy output distribution is balanced into two equal portions.

c) Mean of Maximum (MOM): The Mean of Maximum method first identifies the maximum membership value in the fuzzy output. All output values having this maximum membership are then considered, and their average is taken as the final crisp output. This method focuses only on the values with the highest degree of membership.

d) Smallest of Maximum (SOM): The Smallest of Maximum method identifies all values having the maximum membership degree and selects the smallest of them as the crisp output. It is useful when a lower value is preferred among equally strong fuzzy outputs.

e) Largest of Maximum (LOM): The Largest of Maximum method also identifies the values having maximum membership, but selects the largest value among them. It can be useful when a higher value is preferred when several values have the same maximum membership..

6. Comparison of Defuzzification Methods

The major difference between these methods is the information they consider while producing the crisp output. The **Centroid** and **Bisector** methods consider the overall distribution of the fuzzy output, whereas **MOM, SOM, and LOM** mainly focus on the points having maximum membership. Therefore, the same fuzzy output can produce different crisp values depending on the selected defuzzification method.

7. Mathematical Representation

For a discrete fuzzy output, the Centroid method can be represented as:

Crisp Output = $\Sigma(x_i \times \mu(x_i)) / \Sigma\mu(x_i)$

where x_i represents an output value and $\mu(x_i)$ represents its membership value. The formula calculates a weighted average in which output values with higher membership contribute more strongly to the final result.

For the Mean of Maximum method, if the maximum membership value occurs at multiple output values, the crisp output is obtained by calculating their arithmetic mean.

8. Breast Cancer Dataset

The **Breast Cancer Wisconsin dataset** is a standard dataset available through Scikit-learn. It contains **569 samples, 30 numerical features, and 2 target classes**, representing malignant and benign cases. The features describe characteristics of cell nuclei obtained from digitized images of breast tissue samples. In this experiment, a numerical feature from this dataset can be used as input to demonstrate the generation of fuzzy membership values and the conversion of the resulting fuzzy output into crisp values using different defuzzification techniques.

9. Implementation Methodology

The implementation follows these steps:

1. Load the Breast Cancer dataset using Scikit-learn.
2. Select a suitable numerical feature from the dataset.
3. Normalize or scale the selected feature if required.
4. Define a fuzzy output range and membership function.
5. Generate membership values for the selected data.
6. Apply different defuzzification methods.
7. Calculate the crisp output using Centroid, Bisector, MOM, SOM, and LOM methods.
8. Compare the crisp values obtained from each method.
9. Visualize the fuzzy membership function and defuzzified results using Matplotlib.

10. Applications of Defuzzification

Defuzzification is widely used in applications where fuzzy reasoning must ultimately produce a numerical decision. Some important applications include:

- Medical diagnosis and clinical decision-support systems.
- Temperature and industrial process control.
- Risk assessment and evaluation.
- Robotics and autonomous systems.
- Pattern recognition and classification.
- Image processing.
- Consumer electronics and automation.
- Decision-making systems.

11. Advantages

- Converts fuzzy information into an understandable crisp value.
- Makes fuzzy systems practical for real-world decision-making.
- Provides several methods suitable for different applications.
- Can be implemented using simple mathematical operations.
- Helps integrate fuzzy logic with conventional computational systems. Decision-making systems

12. Limitations

- Different methods may produce different crisp values for the same fuzzy output.
- Selection of a suitable method depends on the requirements of the application.
- Converting a fuzzy distribution into one value can result in some loss of information.
- The result depends on the quality and shape of the membership function.
- Poorly selected membership functions or output ranges can lead to unsuitable decisions

Code & Output: -

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer

# Load Breast Cancer Dataset

data = load_breast_cancer()

X = data.data
y = data.target

print("Dataset Shape:", X.shape)
print("Features:", data.feature_names)

# Select Mean Radius Feature

radius = X[:, 0]

print("\nFirst 10 Radius Values:")
print(radius[:10])

# Fuzzy Membership Function

def high_membership(x):
    if x <= 10:
        return 0
    elif x >= 20:
        return 1
    else:
        return (x - 10) / (20 - 10)

print("\nSample Membership Values:")

for r in [8, 12, 15, 18, 20]:
    print(r, high_membership(r))

# Create Fuzzy Output Set

risk = np.arange(0, 101, 1)

membership = np.zeros(len(risk))

for i, r in enumerate(risk):

    # Rising part
    if 20 <= r <= 50:
        membership[i] = (r - 20) / 30

    # Falling part
    elif 50 < r <= 80:
        membership[i] = (80 - r) / 30

    # Outside fuzzy region
    else:
        membership[i] = 0

# 1. Centroid Method

centroid = np.sum(risk * membership) /
np.sum(membership)

# 2. Bisector Method

total_area = np.sum(membership)

cumulative_area = np.cumsum(membership)

bisector_index = np.argmin(
    np.abs(cumulative_area - total_area / 2)
)

bisector = risk[bisector_index]

# 3. Mean of Maximum (MOM)

max_membership = np.max(membership)

max_values = risk[membership ==
max_membership]

mom = np.mean(max_values)

# 4. Smallest of Maximum (SOM)

som = np.min(max_values)

# 5. Largest of Maximum (LOM)

lom = np.max(max_values)

# Display Results

print("\nDefuzzification Results")
print("----------------------------")

print("Centroid (COG):", round(centroid, 2))
```

```
print("Bisector:", round(bisector, 2))
print("Mean of Maximum (MOM):", round(mom, 2))
print("Smallest of Maximum (SOM):", round(som, 2))
print("Largest of Maximum (LOM):", round(lom, 2))

# Plot Fuzzy Membership Function

plt.figure(figsize=(9, 5))

plt.plot(
    risk,
    membership,
    label="Fuzzy Output Membership"
)

# Defuzzified values

plt.axvline(
    centroid,
    linestyle="--",
    label=f"Centroid = {centroid:.2f}"
)

plt.axvline(
    bisector,
    linestyle="--",
    label=f"Bisector = {bisector:.2f}"
)

plt.axvline(
    mom,
    linestyle="--",
    label=f"MOM = {mom:.2f}"
)

plt.axvline(
    som,
    linestyle="--",
    label=f"SOM = {som:.2f}"
)

plt.axvline(
    lom,
    linestyle="--",
    label=f"LOM = {lom:.2f}"
)

plt.title("Common Defuzzification Methods")
plt.xlabel("Output Value")
plt.ylabel("Membership Value")
plt.legend()
plt.grid(True)

plt.show()
```

Output:

```
Dataset Shape: (569, 30)
Features: ['mean radius' 'mean texture' 'mean perimeter' 'mean area'
 'mean smoothness' 'mean compactness' 'mean concavity'
 'mean concave points' 'mean symmetry' 'mean fractal dimension'
 'radius error' 'texture error' 'perimeter error' 'area error'
 'smoothness error' 'compactness error' 'concavity error'
 'concave points error' 'symmetry error' 'fractal dimension error'
 'worst radius' 'worst texture' 'worst perimeter' 'worst area'
 'worst smoothness' 'worst compactness' 'worst concavity'
 'worst concave points' 'worst symmetry' 'worst fractal dimension']

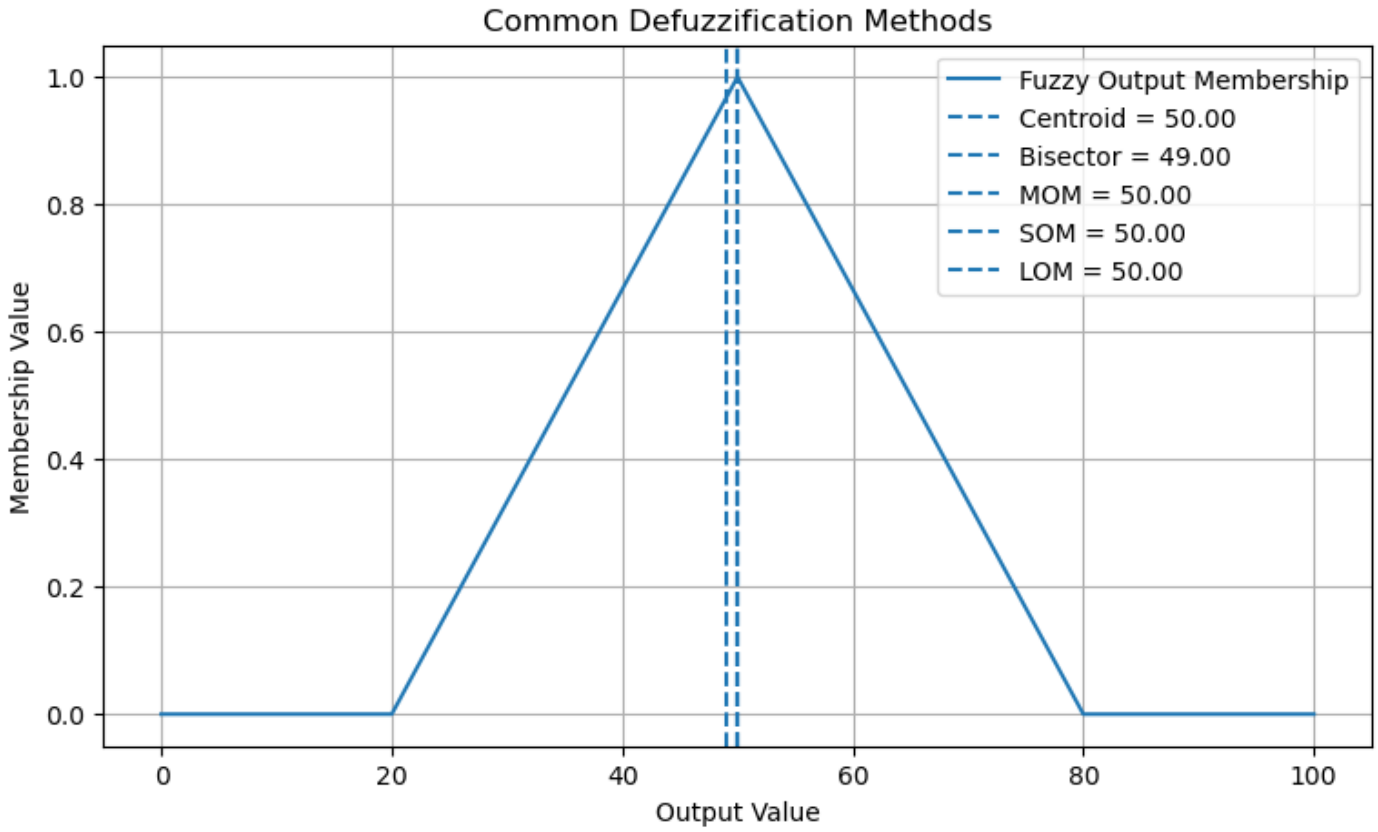
First 10 Radius Values:
[17.99 20.57 19.69 11.42 20.29 12.45 18.25 13.71 13.   12.46]

Sample Membership Values:
8 0
12 0.2
15 0.5
18 0.8
```

20 1

Defuzzification Results

Centroid (COG): 50.0
Bisector: 49
Mean of Maximum (MOM): 50.0
Smallest of Maximum (SOM): 50
Largest of Maximum (LOM): 50



THAKUR COLLEGE OF
ENGINEERING & TECHNOLOGY

Learning Outcomes: The student should have the ability to:

LO 6.1: Understand the concept and importance of defuzzification in fuzzy logic systems.
LO 6.2: Understand and implement common defuzzification methods such as Centroid, Bisector, Mean of Maximum (MOM), Smallest of Maximum (SOM), and Largest of Maximum (LOM).
LO 6.3: Convert fuzzy output membership values into a single crisp numerical value using different defuzzification techniques.
LO 6.4: Compare the results obtained from different defuzzification methods and understand their differences.
LO 6.5: Apply defuzzification techniques to numerical data from the Breast Cancer dataset using Python.

Course Outcome: Apply fuzzy logic concepts and defuzzification techniques to convert fuzzy outputs into crisp values and develop solutions for decision-making problems using Python..

Conclusion:

THAKUR ZCT'S
TCET
THAKUR COLLEGE OF ENGINEERING & TECHNOLOGY

For Faculty Use: -

Correction Parameters	Formative Assessment [40%]	Timely completion of Practical [40%]	Attendance / Learning Attitude [20%]	
Marks Obtained				

Experiment 7 – To Implement Fuzzy Controller System Design. Design a Washing Machine Controller.

Learning Objective: To design and implement a **Fuzzy Logic Controller for a washing machine** that determines the appropriate washing time based on the **dirt level** and **fabric type** using fuzzy membership functions, fuzzy IF–THEN rules, inference, and defuzzification.

Tools: Python (Jupyter Notebook), NumPy, Scikit-Fuzzy and Matplotlib Libraries.

Theory: -

1. Introduction

A **Fuzzy Logic Controller (FLC)** is an intelligent control system that makes decisions using approximate or linguistic information rather than depending only on exact numerical conditions. It is useful for systems where inputs cannot always be represented accurately by simple mathematical equations.

Conventional logic usually represents conditions as either **0 or 1**, whereas fuzzy logic allows intermediate values between 0 and 1. Therefore, concepts such as **low dirt, medium dirt, soft fabric, hard fabric, short washing time,** and **long washing time** can be represented mathematically.

In this experiment, a fuzzy controller is designed for a **washing machine** to automatically determine washing time according to the dirt level of clothes and their fabric characteristics.

2. Fuzzy Logic

Fuzzy Logic is a form of multi-valued logic that represents the degree to which an input belongs to a particular fuzzy set.

The membership value is represented as:

$$0 \leq \mu(x) \leq 10$$

where:

- $\mu(x)=0$ means the element does not belong to the fuzzy set.
- $\mu(x)=1$ means complete membership.
- A value between 0 and 1 represents partial membership.

For example, a dirt level of 8 may partially belong to both the **Medium** and **High** dirt fuzzy sets. This overlapping behaviour provides smoother decisions compared with rigid threshold-based control systems.

3. Fuzzy Logic Controller

A **Fuzzy Logic Controller** converts numerical inputs into fuzzy values, evaluates a collection of fuzzy rules and produces an appropriate numerical output.

The basic operation is:

Crisp Inputs → Fuzzification → Rule Evaluation → Fuzzy Inference → Defuzzification → Crisp Output

For the implemented washing machine controller:

Dirt Level + Fabric Type→Fuzzy Controller→Washing Time

The controller therefore imitates human reasoning. For example:

IF Dirt is High AND Fabric is Hard, THEN Washing Time is Long.

4. Components of Fuzzy Controller

The main components involved in a fuzzy controller are:

1. **Fuzzification:** Converts numerical input values into membership values associated with fuzzy sets.
2. **Membership Functions:** Define how strongly an input belongs to linguistic categories such as Low, Medium or High.
3. **Rule Base:** Contains fuzzy IF–THEN rules representing the control knowledge of the system.
4. **Inference Engine:** Evaluates the applicable rules according to the given input conditions and combines their results.
5. **Defuzzification:** Converts the final fuzzy output into a single numerical value that can be used by the washing machine.

These stages allow the system to convert human-like rules into an actual control output.

5. Input and Output Variables

The implemented fuzzy controller contains **two input variables and one output variable**.

Variable	Type	Range	Fuzzy Sets
Dirt Level	Input	0–10	Low, Medium, High
Fabric Type	Input	0–10	Soft, Normal, Hard
Washing Time	Output	5–40 min	Short, Medium, Long

Dirt Level

It indicates how dirty the clothes are.

- **Low:** Clothes contain less dirt.
- **Medium:** Clothes contain a moderate amount of dirt.
- **High:** Clothes contain a high amount of dirt.

Fabric Type

A numerical scale from 0 to 10 is used to represent the fabric characteristic.

- **Soft**
- **Normal**

- **Hard**

Washing Time

The output represents the duration for which the washing machine should operate.

- **Short**
- **Medium**
- **Long**

6. Membership Functions

The implementation uses **triangular membership functions**, defined using fuzz.trimf().

For Dirt Level:

Low=[0,0,5] Medium=[0,5,10] High=[5,10,10]

For Fabric Type:

Soft=[0,0,5] Normal=[0,5,10] 0] Hard=[5,10,10]

For Washing Time:

Short=[5,5,20] Medium=[10,22.5,35] Long=[25,40,40]

The overlapping membership functions allow a particular numerical value to belong to more than one fuzzy category simultaneously, resulting in gradual and smooth control decisions.

7. Fuzzy Rule Base

The implemented controller consists of **nine fuzzy IF–THEN rules**.

Dirt Level	Fabric Type	Washing Time
Low	Soft	Short
Low	Normal	Short
Low	Hard	Medium
Medium	Soft	Medium
Medium	Normal	Medium
Medium	Hard	Long
High	Soft	Medium
High	Normal	Long
High	Hard	Long

The rules represent the relationship between the washing conditions and the required washing duration.

For example:

- IF **Dirt is Low AND Fabric is Soft**, washing time is **Short**.
- IF **Dirt is Medium AND Fabric is Normal**, washing time is **Medium**.

- IF **Dirt is High AND Fabric is Hard**, washing time is **Long**.

Thus, as the dirt level and fabric requirement increase, the controller generally assigns a longer washing duration.

8. Working of the Implemented Fuzzy Controller

The program performs the following steps:

1. The required Python libraries are imported.
2. Dirt level and fabric type are created as fuzzy **Antecedents**.
3. Washing time is created as a fuzzy **Consequent**.
4. Triangular membership functions are assigned to each variable.
5. Nine fuzzy IF–THEN rules are created.
6. The rules are combined to form the washing machine ControlSystem.
7. Input values for dirt and fabric are supplied.
8. The fuzzy inference process is executed using compute().
9. A crisp washing-time output is obtained.
10. Membership function graphs and the final fuzzy output are displayed using Matplotlib.

This implementation provides a complete fuzzy controller from numerical input to final control decision.

9. Advantages

- Handles uncertainty and approximate input information effectively.
- Does not require a complicated mathematical model of the washing process.
- Provides smooth decisions because fuzzy sets can overlap.
- Human reasoning can easily be represented using IF–THEN rules.
- Automatically adjusts washing time according to different conditions.
- Membership functions and rules can easily be modified according to application requirements.
- Fuzzy controllers are suitable for real-time intelligent control applications.

10. Limitations

- Selection of appropriate membership functions requires domain knowledge.
- Performance depends on the quality of the fuzzy rules.
- Incorrectly designed rules can produce unsuitable control decisions.
- Increasing the number of input variables significantly increases the number of possible rules.
- Membership functions may require repeated tuning.
- A conventional fuzzy controller does not learn automatically from previous washing cycles.

Code & Output: -

Code:

```
import numpy as np
import skfuzzy as fuzz
from skfuzzy import control as ctrl
import matplotlib.pyplot as plt

# FUZZY CONTROLLER FOR WASHING
MACHINE
# Inputs  : Dirt Level, Fabric Type
# Output  : Washing Time
# STEP 1: Define input and output variables

dirt = ctrl.Antecedent(np.arange(0, 10.1, 0.1),
'dirt')
fabric = ctrl.Antecedent(np.arange(0, 10.1, 0.1),
'fabric')
washing_time = ctrl.Consequent(
    np.arange(5, 40.1, 0.1),
    'washing_time'
)

# ----------------------------------------------------
--
# STEP 2: Define membership functions for DIRT
# ----------------------------------------------------
--

dirt['low'] = fuzz.trimf(dirt.universe, [0, 0, 5])
dirt['medium'] = fuzz.trimf(dirt.universe, [0, 5,
10])
dirt['high'] = fuzz.trimf(dirt.universe, [5, 10, 10])

# ----------------------------------------------------
--
# STEP 3: Define membership functions for
FABRIC
# ----------------------------------------------------
--

fabric['soft'] = fuzz.trimf(fabric.universe, [0, 0, 5])
fabric['normal'] = fuzz.trimf(fabric.universe, [0, 5,
10])
fabric['hard'] = fuzz.trimf(fabric.universe, [5, 10,
10])

# ----------------------------------------------------
--
# STEP 4: Define membership functions for
WASHING TIME

# ----------------------------------------------------
--

washing_time['short'] = fuzz.trimf(
    washing_time.universe,
    [5, 5, 20]
)

washing_time['medium'] = fuzz.trimf(
    washing_time.universe,
    [10, 22.5, 35]
)

washing_time['long'] = fuzz.trimf(
    washing_time.universe,
    [25, 40, 40]
)

# ----------------------------------------------------
--
# STEP 5: Define fuzzy rules
# ----------------------------------------------------
--

rule1 = ctrl.Rule(
    dirt['low'] & fabric['soft'],
    washing_time['short']
)

rule2 = ctrl.Rule(
    dirt['low'] & fabric['normal'],
    washing_time['short']
)

rule3 = ctrl.Rule(
    dirt['low'] & fabric['hard'],
    washing_time['medium']
)

rule4 = ctrl.Rule(
    dirt['medium'] & fabric['soft'],
    washing_time['medium']
)

rule5 = ctrl.Rule(
    dirt['medium'] & fabric['normal'],
    washing_time['medium']
)
```

```
rule6 = ctrl.Rule(
    dirt['medium'] & fabric['hard'],
    washing_time['long']
)

rule7 = ctrl.Rule(
    dirt['high'] & fabric['soft'],
    washing_time['medium']
)

rule8 = ctrl.Rule(
    dirt['high'] & fabric['normal'],
    washing_time['long']
)

rule9 = ctrl.Rule(
    dirt['high'] & fabric['hard'],
    washing_time['long']
)

# ------------------------------------------------------
--
# STEP 6: Create fuzzy control system
# ------------------------------------------------------
--

washing_control = ctrl.ControlSystem([
    rule1,
    rule2,
    rule3,
    rule4,
    rule5,
    rule6,
    rule7,
    rule8,
    rule9
])

washing_machine =
ctrl.ControlSystemSimulation(washing_control)


dirt_value = 8
fabric_value = 8

washing_machine.input['dirt'] = dirt_value
washing_machine.input['fabric'] = fabric_value

# ------------------------------------------------------
--
# STEP 8: Perform fuzzy inference
# ------------------------------------------------------
--

washing_machine.compute()

# ------------------------------------------------------
--
# STEP 9: Display output
# ------------------------------------------------------
--

result = washing_machine.output['washing_time']

print("=" * 50)
print("      FUZZY WASHING MACHINE
CONTROLLER")
print("=" * 50)

print(f"Dirt Level   : {dirt_value}/10")
print(f"Fabric Type  : {fabric_value}/10")
print(f"Washing Time : {result:.2f} minutes")

print("=" * 50)

# ------------------------------------------------------
--
# STEP 10: Display membership functions
# ------------------------------------------------------
--

dirt.view()
plt.title("Dirt Level Membership Functions")
plt.show()

fabric.view()
plt.title("Fabric Type Membership Functions")
plt.show()

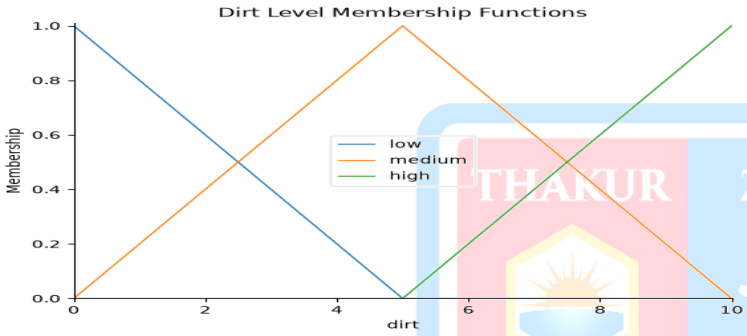
washing_time.view()
plt.title("Washing Time Membership Functions")
plt.show()

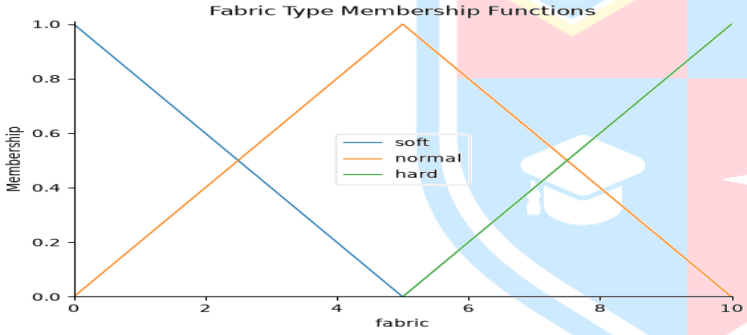


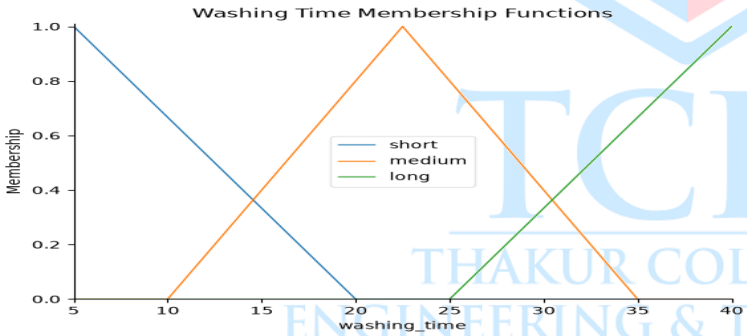
washing_time.view(sim=washing_machine)
plt.title("Fuzzy Controller Output")
plt.show()
```

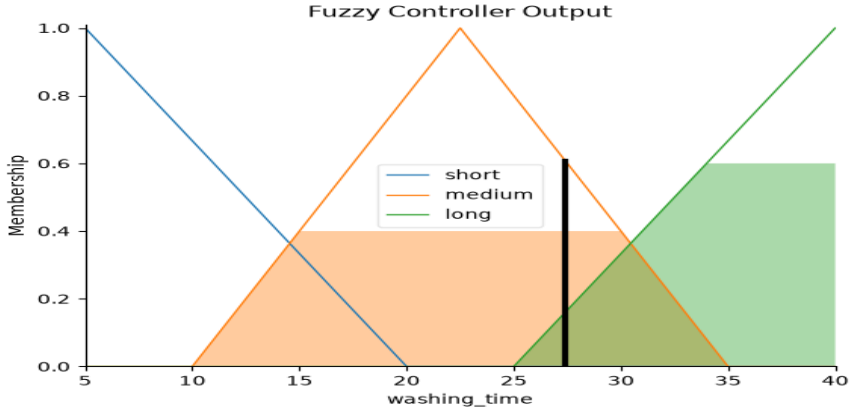
Output:

```
==================================================
      FUZZY WASHING MACHINE CONTROLLER
==================================================
Dirt Level   : 8/10
Fabric Type  : 8/10
Washing Time : 27.41 minutes
==================================================
```









Learning Outcomes: The student should have the ability to:
LO 7.1: Understand the architecture and working of a Fuzzy Logic Controller for an intelligent control application.
LO 7.2: Design fuzzy input and output variables using suitable triangular membership functions.
LO 7.3: Implement fuzzy IF–THEN rules for controlling washing time based on dirt level and fabric type.
LO 7.4: Perform fuzzy inference and defuzzification to obtain a crisp washing-time output and visualize the corresponding membership functions.

Course Outcome: Design, implement, and evaluate a Fuzzy Logic Controller using suitable membership functions and fuzzy rules to provide intelligent control for real-world applications.

Conclusion:

THAKUR ZCT'S TM
TCET
THAKUR COLLEGE OF ENGINEERING & TECHNOLOGY

For Faculty Use: -

Correction Parameters	Formative Assessment [40%]	Timely completion of Practical [40%]	Attendance / Learning Attitude [20%]	
Marks Obtained				

Experiment 8 – To Apply Genetic Algorithm Operators on a Given Problem.

Learning Objective: To understand and implement Genetic Algorithm (GA) operators, including selection, crossover, and mutation, on a given problem to generate improved solutions through iterative evolution and optimization.

Tools: Python (Jupyter Notebook), NumPy, Matplotlib Libraries.

Theory: -

1. Introduction

A Genetic Algorithm (GA) is an optimization and search technique inspired by the process of natural selection and the theory of biological evolution. It is used to find approximate solutions to complex problems where traditional optimization methods may be difficult to apply.

Genetic Algorithms work with a population of potential solutions rather than a single solution. Each solution is represented as a chromosome, which consists of genes. The quality of each solution is evaluated using a fitness function. Solutions with better fitness are more likely to be selected for reproduction, allowing improved solutions to emerge over successive generations.

In this experiment, Genetic Algorithm operators are applied to a given problem to understand how selection, crossover, and mutation contribute to the generation of new solutions and the optimization process.

2. Genetic Algorithm

A Genetic Algorithm is a population-based search algorithm that repeatedly improves a set of candidate solutions using principles inspired by biological evolution.

The main objective is to identify an optimal or near-optimal solution by evolving the population over multiple generations.

The basic process of a Genetic Algorithm is:

Initialization → Fitness Evaluation → Selection → Crossover → Mutation → Replacement → Termination

The process continues until a suitable solution is obtained or a specified termination condition is satisfied.

3. Basic Terminology of Genetic Algorithm

The important terms used in Genetic Algorithms are:

1. Population: A collection of candidate solutions considered during a particular generation.
2. Chromosome: A representation of an individual solution to the given problem.
3. Gene: An individual component of a chromosome representing a particular characteristic of a solution.
4. Fitness Function: A function used to measure the quality of a candidate solution.
5. Generation: One iteration of the evolutionary process in which the population is evaluated and new solutions are produced.

6. Parent: An individual selected from the population to participate in reproduction.
7. Offspring: A new individual generated by applying genetic operators to selected parents.
8. Termination Condition: A condition that determines when the algorithm should stop, such as reaching a maximum number of generations or obtaining a satisfactory solution.

4. Genetic Algorithm Operators

Genetic Algorithm operators are the fundamental mechanisms responsible for evolving a population and generating improved solutions. The three principal operators are selection, crossover, and mutation.

4.1 Selection Operator

The selection operator chooses individuals from the current population to act as parents for producing the next generation. The selection process generally considers the fitness values of individuals, giving better-performing solutions a higher probability of being selected.

Common selection techniques include:

- Roulette Wheel Selection: Individuals are selected with probabilities proportional to their fitness values.
- Tournament Selection: A group of individuals is selected randomly, and the individual with the best fitness among them is chosen.
- Rank Selection: Individuals are ranked according to fitness, and selection probabilities are assigned based on their ranks.

Selection helps the algorithm preserve and propagate useful characteristics across generations. However, excessive selection pressure may reduce population diversity and cause premature convergence.

4.2 Crossover Operator

The crossover operator combines genetic information from two selected parent chromosomes to produce one or more offspring. It enables the algorithm to explore new combinations of characteristics that may lead to better solutions.

Common crossover techniques include:

- Single-Point Crossover: A crossover point is selected, and the chromosome segments after that point are exchanged between two parents.
- Two-Point Crossover: Two crossover points are selected, and the chromosome segments between those points are exchanged.
- Uniform Crossover: Each gene is independently selected from either parent according to a predefined probability.

For example, consider two binary parent chromosomes:

Parent 1: `110|010`

Parent 2: `001|111`

After single-point crossover, the offspring may be:

Offspring 1: `110111`

Offspring 2: `001010`

Crossover combines information from existing solutions and can produce offspring with potentially improved fitness.

4.3 Mutation Operator

The mutation operator introduces random changes to one or more genes in a chromosome. Its primary purpose is to maintain genetic diversity and introduce new characteristics into the population.

For binary chromosomes, mutation is commonly performed by flipping a randomly selected bit from 0 to 1 or from 1 to 0.

For example:

Before mutation: `110010`

After mutation: `110110`

Here, the fourth bit changes from 0 to 1.

Mutation helps the algorithm explore new regions of the search space and reduces the likelihood of becoming trapped in a local optimum. However, an excessively high mutation rate may disrupt useful genetic information.

5. Fitness Function

The fitness function evaluates how well a candidate solution satisfies the objective of the given problem. It assigns a numerical fitness value to each individual in the population.

The selection operator uses these fitness values to determine which individuals are more likely to reproduce.

For a maximization problem, a higher fitness value generally indicates a better solution. For a minimization problem, the fitness evaluation or selection procedure must be designed appropriately so that solutions with lower objective values are favored.

The design of the fitness function depends on the nature and requirements of the optimization problem.

6. Working of a Genetic Algorithm

The Genetic Algorithm follows these steps:

1. Initialize Population: Generate an initial population of candidate solutions, usually randomly.
2. Evaluate Fitness: Calculate the fitness value of each individual using the fitness function.
3. Select Parents: Choose individuals from the population using a suitable selection operator.
4. Apply Crossover: Combine selected parents to produce offspring.

5. Apply Mutation: Introduce random changes into selected offspring according to the mutation probability.
6. Form the New Population: Replace some or all of the existing individuals with the newly generated offspring. An elitism strategy may preserve the best individuals.
7. Check Termination: Determine whether the stopping condition has been satisfied.
8. Repeat: If the termination condition is not satisfied, evaluate the new population and repeat the evolutionary process.

Through repeated application of these steps, the population may gradually develop solutions with improved fitness.

7. Advantages of Genetic Algorithms

- Suitable for solving complex optimization and search problems.
- Can explore a large search space without requiring gradient information.
- Work with a population of candidate solutions rather than a single solution.
- Crossover combines useful characteristics from different individuals.
- Mutation maintains diversity and supports exploration of new solutions.
- Can be applied to discrete, binary, and appropriately encoded continuous optimization problems.
- The algorithm can be adapted to different problems by modifying the chromosome representation, fitness function, and genetic operators.

8. Limitations of Genetic Algorithms

- There is no guarantee of finding the global optimum.
- Performance depends on population size, crossover rate, mutation rate, and selection strategy.
- Fitness evaluation may be computationally expensive for complex problems.
- Poorly designed fitness functions can produce unsuitable solutions.
- Excessive selection pressure may cause premature convergence.
- High mutation rates may destroy useful genetic information.
- Appropriate parameter selection may require experimentation and tuning.

TCET
THAKUR COLLEGE OF
ENGINEERING & TECHNOLOGY

Code & Output: -

Code:

```
import random
# Genetic Algorithm: Applying GA Operators
POPULATION_SIZE = 6
CHROMOSOME_LENGTH = 5
GENERATIONS = 10
CROSSOVER_RATE = 0.8
MUTATION_RATE = 0.1

# Step 1: Initialize population
def create_population():
    return [
        [random.randint(0, 1) for _ in
range(CHROMOSOME_LENGTH)]
        for _ in range(POPULATION_SIZE)
    ]

# Convert binary chromosome to decimal
def decode(chromosome):
    return int("".join(map(str, chromosome)), 2)

# Step 2: Calculate fitness
def fitness(chromosome):
    x = decode(chromosome)
    return x ** 2

# Step 3: Tournament selection
def selection(population):
    candidates = random.sample(population, 3)
    return max(candidates, key=fitness)[:]

# Step 4: Single-point crossover
def crossover(parent1, parent2):
    if random.random() < CROSSOVER_RATE:
        point = random.randint(1,
CHROMOSOME_LENGTH - 1)
        child1 = parent1[:point] + parent2[point:]
        child2 = parent2[:point] + parent1[point:]
        return child1, child2
    return parent1[:], parent2[:]

# Step 5: Bit-flip mutation
def mutation(chromosome):
    for i in range(CHROMOSOME_LENGTH):
        if random.random() < MUTATION_RATE:
            chromosome[i] = 1 - chromosome[i]
    return chromosome

# Step 6: Run Genetic Algorithm
population = create_population()

print("Initial Population:")
for individual in population:
    print(individual, "Fitness:", fitness(individual))

for generation in range(1, GENERATIONS + 1):
    new_population = []

    while len(new_population) <
POPULATION_SIZE:
        parent1 = selection(population)
        parent2 = selection(population)

        child1, child2 = crossover(parent1, parent2)

        new_population.append(mutation(child1))
        if len(new_population) <
POPULATION_SIZE:
            new_population.append(mutation(child2))

    population = new_population

    best = max(population, key=fitness)
    print(
        f"Generation {generation}: "
        f"Best x = {decode(best)}, "
        f"Fitness = {fitness(best)}"
    )

# Step 7: Display best solution
best = max(population, key=fitness)

print("\nFinal Result")
print("Best Chromosome:", best)
print("Best Solution (x):", decode(best))
print("Maximum Fitness:", fitness(best))
```

Output:

Initial Population:
[1, 0, 1, 1, 0] Fitness: 484
[0, 1, 0, 1, 1] Fitness: 121
[1, 1, 0, 0, 1] Fitness: 625
[0, 0, 1, 1, 0] Fitness: 36
[1, 0, 0, 1, 1] Fitness: 361
[0, 1, 1, 1, 1] Fitness: 225

Generation 1: Best x = 27, Fitness = 729
Generation 2: Best x = 29, Fitness = 841
Generation 3: Best x = 29, Fitness = 841
Generation 4: Best x = 30, Fitness = 900
Generation 5: Best x = 30, Fitness = 900
Generation 6: Best x = 31, Fitness = 961
Generation 7: Best x = 31, Fitness = 961
Generation 8: Best x = 31, Fitness = 961
Generation 9: Best x = 31, Fitness = 961
Generation 10: Best x = 31, Fitness = 961

Final Result
Best Chromosome: [1, 1, 1, 1, 1]
Best Solution (x): 31
Maximum Fitness: 961

THAKUR ZCT'S
TCET
THAKUR COLLEGE OF
ENGINEERING & TECHNOLOGY

Learning Outcomes: The student should have the ability to:
LO 8.1: Understand the basic concepts and working principles of Genetic Algorithms for solving optimization problems.
LO 8.2: Apply selection operators to choose suitable individuals from a population based on their fitness values.
LO 8.3: Implement crossover and mutation operators to generate new solutions and maintain genetic diversity in the population.
LO 8.4: Evaluate the effect of Genetic Algorithm operators on population evolution and the quality of solutions for a given problem.

Course Outcome: Design, implement, and evaluate Genetic Algorithm operators, including selection, crossover, and mutation, to solve optimization problems and improve solution quality through evolutionary computation.

Conclusion:

THAKUR ZCT'S
TCET
THAKUR COLLEGE OF ENGINEERING & TECHNOLOGY

For Faculty Use: -

Correction Parameters	Formative Assessment [40%]	Timely completion of Practical [40%]	Attendance / Learning Attitude [20%]	
Marks Obtained				

Experiment 9 – Case Study on ANFIS Hybrid Systems and Study of the Designing of Neuro-Fuzzy Systems.

Learning Objective: To study the architecture and working principles of Adaptive Neuro-Fuzzy Inference Systems (ANFIS) and understand the design of Neuro-Fuzzy Systems by integrating artificial neural networks with fuzzy logic for intelligent decision-making, modeling, and problem-solving under uncertainty.

Tools: Python (Jupyter Notebook), NumPy, Scikit-Fuzzy, Scikit-Learn, and Matplotlib Libraries.

Theory: -

1. Introduction

Soft Computing is a branch of computing that deals with complex problems involving uncertainty, imprecision, ambiguity, and nonlinear relationships. Unlike conventional computing techniques that generally depend on precise mathematical models and exact input values, soft computing techniques can handle approximate information and learn from available data.

Two important techniques used in soft computing are Artificial Neural Networks (ANNs) and Fuzzy Logic Systems (FLSs). Artificial Neural Networks learn patterns and relationships from data, whereas fuzzy logic systems represent knowledge using linguistic variables and fuzzy IF–THEN rules.

However, each technique has certain limitations when used independently. Neural networks may behave like black-box models and may not provide easily interpretable reasoning. Fuzzy systems can represent human knowledge effectively, but designing suitable membership functions and fuzzy rules may require considerable expert knowledge.

To overcome these limitations, Neuro-Fuzzy Systems combine the learning capabilities of neural networks with the reasoning capabilities of fuzzy logic. An important example is the Adaptive Neuro-Fuzzy Inference System (ANFIS), which uses an adaptive network to tune the parameters of a fuzzy inference system.

This case study examines ANFIS hybrid systems, their architecture, working principles, applications, advantages, and limitations. It also studies the design of Neuro-Fuzzy Systems and explains how neural learning and fuzzy reasoning can be integrated to develop intelligent decision-making models.

2. Fundamentals of Fuzzy Logic Systems

Fuzzy Logic is a mathematical approach that represents uncertainty through degrees of membership rather than restricting membership to only true or false.

In classical Boolean logic, a statement can have a truth value of either 0 or 1. In fuzzy logic, the degree of membership of an element in a fuzzy set can take any value between 0 and 1.

The membership function is represented as:

$$\mu_A(x) \in [0,1]$$

where:

- $\mu_A(x)$ is the membership degree of element x in fuzzy set A.
- A membership value of 0 indicates no membership.
- A membership value of 1 indicates complete membership.
- A value between 0 and 1 indicates partial membership.

For example, in an air-conditioning system, a temperature of $28°C$ may belong partly to the fuzzy sets Warm and Hot. This overlapping representation allows the system to make gradual decisions rather than relying on rigid temperature thresholds.

2.1 Components of a Fuzzy Inference System

A Fuzzy Inference System (FIS) generally consists of the following components:

1. Fuzzification: Converts crisp numerical inputs into their corresponding membership degrees.
2. Knowledge Base: Stores the membership function definitions and fuzzy IF–THEN rules.
3. Inference Engine: Evaluates the rules and combines their outputs to determine the fuzzy response.
4. Defuzzification: Converts the resulting fuzzy output into a crisp numerical value when required.

For example, a temperature-control system may use the rule:

IF temperature is High, THEN cooling power is Strong.

A fuzzy system evaluates the degree to which the temperature is High and uses this degree to determine the appropriate cooling response.

3. Fundamentals of Artificial Neural Networks

An Artificial Neural Network is a computational model inspired by the interconnected structure of biological neurons. It consists of processing units called neurons, organized into input, hidden, and output layers.

Neural networks learn relationships between inputs and outputs by adjusting connection weights and biases during training.

The output of a typical neuron can be represented as:

$$y = f\left(\sum_{i=1}^{n} w_i x_i + b\right)$$

where:

- x_i represents an input feature.
- w_i represents the corresponding connection weight.
- b is the bias.
- f is an activation function.
- y is the neuron output.

During training, the network compares its predictions with the target values and adjusts its parameters to reduce prediction error.

The major advantages of neural networks include learning from examples, modeling nonlinear relationships, and adapting to complex data. However, their internal computations may be difficult to interpret, and they generally require representative training data.

4. Neuro-Fuzzy Systems

A Neuro-Fuzzy System is a hybrid intelligent system that combines fuzzy logic with neural network learning mechanisms. It uses fuzzy sets and rules to represent knowledge while employing learning algorithms to adjust system parameters automatically.

The main objective is to combine the interpretability of fuzzy reasoning with the adaptive learning capability of neural networks.

A Neuro-Fuzzy System can be represented as:

Input Data → Fuzzification → Fuzzy Rule Evaluation → Neural Learning and Parameter Adaptation → Output

The exact architecture depends on the chosen Neuro-Fuzzy model. In some systems, neural learning is used to tune membership functions; in others, it also adjusts rule weights or other model parameters.

4.1 Need for Neuro-Fuzzy Integration

A conventional fuzzy system depends significantly on the quality of its membership functions and rules. If the system designer selects inappropriate parameters, the output may not represent the actual behavior of the system accurately.

A neural network can learn patterns from numerical data, but its internal knowledge is not always represented as understandable linguistic rules.

Neuro-Fuzzy integration addresses these challenges through the following mechanisms:

- Learning: Model parameters can be adjusted using training data.
- Adaptation: The system can improve its behavior as it learns from examples.
- Nonlinear modeling: Complex input-output relationships can be approximated.
- Knowledge representation: Fuzzy IF–THEN rules provide a structured representation of relationships.
- Reduced manual tuning: Some membership function and rule-related parameters can be optimized automatically.

Thus, Neuro-Fuzzy Systems are useful when a problem requires both data-driven learning and interpretable fuzzy reasoning.

5. Adaptive Neuro-Fuzzy Inference System (ANFIS)

ANFIS stands for Adaptive Neuro-Fuzzy Inference System. It is a hybrid intelligent modeling technique that combines a fuzzy inference system with an adaptive network structure.

ANFIS was introduced by Jang in 1993. It commonly implements a first-order Sugeno-type fuzzy inference system, in which each rule produces a constant or linear function of the input variables.

The adaptive network contains adjustable parameters associated with fuzzy membership functions and rule consequents. These parameters can be tuned using training data to minimize the difference between predicted and target outputs.

ANFIS is particularly suitable for nonlinear function approximation, prediction, classification-related modeling, and control applications where input-output relationships are difficult to express through a precise mathematical model.

5.1 Sugeno-Type Fuzzy Rules

Consider a system with two input variables, x and y, and an output z. A first-order Sugeno model may use the following two rules:

Rule 1:

IF x is A_1 AND y is B_1, THEN

$$f_1 = p_1 x + q_1 y + r_1$$

Rule 2:

IF x is A_2 AND y is B_2, THEN

$$f_2 = p_2 x + q_2 y + r_2$$

Here:

- A_i and B_i are fuzzy sets associated with the input variables.
- p_i, q_i, and r_i are consequent parameters.
- f_i is the output function of the corresponding rule.

The firing strength of each rule depends on the membership degrees of the input values. These firing strengths determine how strongly each rule contributes to the final output.

5.2 Architecture of ANFIS

A typical ANFIS architecture consists of five computational layers.

Layer 1 — Fuzzification

Calculates membership degrees for the input variables.

Layer 2 — Rule Evaluation

Calculates the firing strength of each fuzzy rule.

Layer 3 — Normalization

Normalizes the firing strengths of the rules.

Layer 4 — Consequent Calculation

Computes each rule's weighted output using its consequent function.

Layer 5 — Output Aggregation

Combines the weighted rule outputs to produce the final prediction.

The five layers operate as follows.

Layer 1: Fuzzification Layer

Each node calculates the membership degree of an input variable in a fuzzy set. For example:

$$O_{1,i} = \mu_{A_i}(x)$$

where $O_{1,i}$ is the output of node i, and $\mu_{A_i}(x)$ is the membership function.

The parameters controlling the shape and position of the membership functions are called premise parameters.

Layer 2: Rule Layer

Each node represents a fuzzy rule and calculates its firing strength. For an AND operator implemented using multiplication:

$$w_i = \mu_{A_i}(x)\mu_{B_i}(y)$$

A higher firing strength indicates that the input satisfies the conditions of the corresponding rule to a greater degree.

Layer 3: Normalization Layer

The firing strengths are normalized to determine the relative contribution of each rule:

$$\bar{w}_i = \frac{w_i}{\sum_j w_j}$$

The normalized firing strengths generally sum to 1, provided the total firing strength is nonzero.

Layer 4: Consequent Layer

Each node computes the weighted output of its corresponding rule:

$$O_{4,i} = \bar{w}_i f_i$$

For a first-order Sugeno system, f_i is a linear function of the input variables.

Layer 5: Output Layer

The final output is obtained by adding the weighted outputs of all rules:

$$z = \sum_i \bar{w}_i \, f_i$$

This produces a crisp numerical prediction from the fuzzy rule system.

5.3 Learning Mechanism of ANFIS

ANFIS commonly uses a hybrid learning algorithm to adjust its parameters.

1. Forward pass: The premise parameters are held fixed, and the consequent parameters are estimated using least-squares estimation.
2. Backward pass: The consequent parameters are held fixed, and the premise parameters are adjusted using gradient-based learning.
3. Error evaluation: The predicted output is compared with the target output to calculate the training error.
4. Parameter update: The learning process adjusts the model parameters to reduce the error.
5. Iteration: The forward and backward passes are repeated until a stopping criterion is reached.

A commonly used error measure for regression is Mean Squared Error (MSE):

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

where y_i is the actual target, $\hat{y}_i$ is the predicted value, and N is the number of observations.

A lower MSE indicates smaller average squared prediction errors on the evaluated dataset. However, training error alone does not establish that a model generalizes well to unseen data.

6. Case Study: ANFIS-Based Temperature Prediction

6.1 Problem Statement

Temperature prediction is an example of a nonlinear modeling problem in which environmental variables may have complex relationships. An ANFIS model can be designed to estimate temperature using relevant input measurements and historical observations.

For this illustrative case study, consider a system that uses humidity and an additional environmental measurement as inputs and predicts temperature as its output. The second input could be selected according to the available dataset, such as a previous temperature reading.

The purpose is to demonstrate how fuzzy rules and data-driven parameter learning can be combined to build a predictive model.

This is a proposed case-study example, not a claim that a model was trained or experimentally evaluated.

6.2 Input and Output Variables

Variable	Role	Description
Humidity	Input	Represents the moisture level in the environment
Previous temperature or another relevant measurement	Input	Provides additional environmental information
Predicted temperature	Output	Estimated temperature for the target observation

The precise input variables, their ranges, and the dataset must be determined before implementation.

6.3 Fuzzy Rule Design

An initial fuzzy rule base can represent qualitative relationships between the inputs and predicted temperature.

Examples of possible rules are:

- IF humidity is Low AND the previous temperature is Low, THEN predicted temperature is Low.
- IF humidity is Medium AND the previous temperature is Medium, THEN predicted temperature is Medium.
- IF humidity is High AND the previous temperature is High, THEN predicted temperature is High.

These rules illustrate the structure of a fuzzy model. Their actual validity depends on the dataset and the physical relationships represented by the problem.

In ANFIS, the initial membership functions and consequent parameters can be tuned using training examples rather than relying entirely on manually selected values.

6.4 Model Training and Evaluation

A practical implementation would involve the following steps:

1. Collect a dataset containing the input measurements and corresponding target temperatures.
2. Remove or handle missing values and examine the input distributions.
3. Divide the dataset into training and testing subsets, maintaining appropriate separation between observations.
4. Normalize or scale variables if required by the chosen implementation.
5. Define the Sugeno-type fuzzy inference system and initialize its membership functions.
6. Train the ANFIS model using an appropriate learning algorithm.
7. Generate temperature predictions for the testing dataset.
8. Evaluate predictive performance using metrics such as MSE, Root Mean Squared Error (RMSE), and Mean Absolute Error (MAE).
9. Compare the predictions with actual values and inspect the residual errors.

The metrics are defined as follows:

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}$$

$$\text{MAE} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|$$

The actual performance of the model cannot be determined without running the implementation on a suitable dataset. Therefore, no numerical accuracy or error value is assumed in this case study.

6.5 Expected Role of ANFIS

ANFIS provides a framework for learning nonlinear relationships between environmental inputs and the target temperature. Its fuzzy rules offer an interpretable representation of input-output behavior, while its adaptive parameters allow the model to fit the training data.

Its usefulness must be established through comparison with appropriate baseline models and evaluation on unseen data. The quality of the prediction depends on dataset representativeness, input selection, model complexity, and training configuration.

7. Design of Neuro-Fuzzy Systems

Designing a Neuro-Fuzzy System involves selecting a suitable fuzzy inference structure, defining the input variables, establishing an initial rule base, and choosing an appropriate learning algorithm.

The general design procedure is as follows.

Step 1: Define the Problem

Identify the objective of the system, the inputs available, the required output, and the constraints of the application.

Step 2: Collect and Prepare Data

Collect representative examples and perform data cleaning, missing-value handling, feature selection, and suitable preprocessing.

Step 3: Select the Neuro-Fuzzy Architecture

Choose an architecture appropriate for the task. ANFIS is suitable for many supervised input-output modeling problems, particularly when a Sugeno-type inference system is appropriate.

Step 4: Define Input and Output Variables

Determine the input features, output variables, numerical ranges, and any required scaling.

Step 5: Initialize Membership Functions

Select suitable membership function types, such as triangular, trapezoidal, or Gaussian functions. Initial parameters can be based on expert knowledge, data distributions, or clustering methods.

Step 6: Construct the Fuzzy Rule Base

Develop fuzzy IF–THEN rules that represent relationships between the input conditions and outputs. The number of rules should be appropriate for the problem complexity and available data.

Step 7: Train the Model

Apply an appropriate learning algorithm to adjust the premise and consequent parameters using training data.

Step 8: Validate the Model

Evaluate performance using validation data and monitor for overfitting. Modify the model structure or parameters if required.

Step 9: Test and Compare

Assess the final model on held-out test data and compare its performance with suitable baseline methods.

Step 10: Deploy and Monitor

Integrate the model into the intended application, monitor its predictions, and retrain or recalibrate it when changes in the data or operating conditions justify doing so.

8. Applications of Neuro-Fuzzy Systems

Neuro-Fuzzy Systems and ANFIS are used in several application areas.

1. Industrial Process Control: Modeling and controlling nonlinear processes such as temperature, pressure, and flow.
2. Forecasting: Estimating future values using historical observations and related input variables.
3. Medical Decision Support: Supporting predictive modeling from patient measurements, subject to appropriate clinical validation.
4. Power Systems: Modeling electricity demand, forecasting load, and supporting control decisions.
5. Agriculture: Estimating crop-related variables and supporting irrigation or environmental management.
6. Robotics: Supporting control and decision-making in systems with uncertain or nonlinear behavior.
7. Fault Diagnosis: Identifying abnormal operating conditions by learning patterns from system measurements.
8. Pattern Recognition: Modeling complex relationships among features for classification-related tasks.

The suitability of a Neuro-Fuzzy System depends on the available data, the problem structure, computational requirements, and the level of interpretability needed.

9. Advantages of ANFIS and Neuro-Fuzzy Systems

- Combine fuzzy reasoning with data-driven learning.

- Model nonlinear input-output relationships.
- Automatically tune selected membership function and rule-consequent parameters.
- Represent system behavior through fuzzy rules.
- Support prediction, modeling, and control applications.
- Reduce the need to determine every model parameter manually.
- Provide a structured framework for combining expert knowledge with experimental observations.

10. Limitations of ANFIS and Neuro-Fuzzy Systems

- The number of fuzzy rules can grow rapidly as the number of input variables and membership functions increases.
- Model performance depends on the quality and representativeness of the training data.
- Training can become computationally expensive for complex models.
- Poorly initialized membership functions may affect convergence and prediction quality.
- Excessive model complexity can cause overfitting.
- ANFIS performance is sensitive to architectural choices and training parameters.
- The presence of fuzzy rules does not automatically guarantee that the entire model is easy to interpret.
- Generalization to unseen conditions must be established through independent evaluation.
- Some ANFIS implementations are less convenient for very high-dimensional problems than alternative machine-learning architectures.

11. Conclusion of the Case Study

Neuro-Fuzzy Systems integrate the learning capabilities of artificial neural networks with the reasoning mechanisms of fuzzy logic. ANFIS is an important example that uses an adaptive network to implement and optimize a fuzzy inference system, commonly based on the Sugeno model.

The five-layer ANFIS architecture performs fuzzification, rule evaluation, normalization, consequent calculation, and output aggregation. Its hybrid learning mechanism adjusts premise and consequent parameters using training data to improve the model's predictions.

The illustrative temperature-prediction case study demonstrates how an ANFIS model can be structured, trained, and evaluated for a nonlinear prediction problem. Actual predictive performance must be established experimentally using a suitable dataset and independent evaluation. Overall, Neuro-Fuzzy Systems provide a useful approach to intelligent modeling when both adaptive learning and fuzzy knowledge representation are required.

Code & Output: -
Code

```
import numpy as np
import matplotlib.pyplot as plt

# ANFIS: Adaptive Neuro-Fuzzy Inference
System
# Case Study: Temperature Prediction

np.random.seed(42)

# Step 1: Generate sample training data
humidity = np.linspace(20, 90, 100)
previous_temp = np.linspace(15, 40, 100)

X = np.column_stack((humidity, previous_temp))

# Illustrative target relationship (synthetic data)
y = 0.15 * humidity + 0.65 * previous_temp + 2

# Step 2: Define Gaussian membership functions
centers = np.array([
    [20, 15], [20, 40],
    [90, 15], [90, 40]
], dtype=float)

sigmas = np.array([25.0, 10.0])

def membership(x, center, sigma):
    return np.exp(-((x - center) ** 2) / (2 * sigma
** 2))

# Step 3: Calculate normalized rule firing
strengths
def get_strengths(X):
    strengths = []

    for c in centers:
        mu1 = membership(X[:, 0], c[0], sigmas[0])
        mu2 = membership(X[:, 1], c[1], sigmas[1])
        strengths.append(mu1 * mu2)
    strengths = np.array(strengths).T
    return strengths / (strengths.sum(axis=1,
keepdims=True) + 1e-10)

# Step 4: Train Sugeno consequent parameters
# Each rule has the form:
# f_i = p_i * humidity + q_i * previous_temp + r_i
def design_matrix(X, strengths):
    blocks = []

    for i in range(strengths.shape[1]):
        w = strengths[:, i:i+1]
        blocks.append(w * X[:, 0:1])
        blocks.append(w * X[:, 1:2])
        blocks.append(w)

    return np.hstack(blocks)

strengths = get_strengths(X)
A = design_matrix(X, strengths)

# Least-squares estimation of consequent
parameters
theta, _, _, _ = np.linalg.lstsq(A, y, rcond=None)

# Step 5: Predict output
def predict(X):
    strengths = get_strengths(X)
    A = design_matrix(X, strengths)
    return A @ theta

predicted = predict(X)

# Step 6: Evaluate model
mse = np.mean((y - predicted) ** 2)
rmse = np.sqrt(mse)
mae = np.mean(np.abs(y - predicted))

print("ANFIS TEMPERATURE PREDICTION")
print("-" * 35)
print("Training samples:", len(X))
print("Number of fuzzy rules:", len(centers))
print("Mean Squared Error:", round(mse, 4))
print("Root Mean Squared Error:", round(rmse,
4))
print("Mean Absolute Error:", round(mae, 4))

# Step 7: Predict for a sample input
sample = np.array([[60, 30]], dtype=float)
result = predict(sample)[0]

print("\nSample Input:")
print("Humidity:", sample[0, 0], "%")
print("Previous Temperature:", sample[0, 1], "C")
```

```
print("Predicted Temperature:", round(result, 2), "C")

# Step 8: Plot actual and predicted outputs
plt.figure(figsize=(8, 5))
plt.plot(y, label="Target Output")
plt.plot(predicted, "--", label="ANFIS Prediction")
plt.xlabel("Sample Index")
plt.ylabel("Temperature (illustrative units)")
plt.title("ANFIS Model: Target vs Predicted Output")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

Output:

```
ANFIS TEMPERATURE PREDICTION
---------------------------------
Training samples: 100
Number of fuzzy rules: 4
Mean Squared Error: 0.0
Root Mean Squared Error: 0.0
Mean Absolute Error: 0.0

Sample Input:
Humidity: 60.0 %
Previous Temperature: 30.0 C
Predicted Temperature: 30.5 C
```

TCET
THAKUR COLLEGE OF
ENGINEERING & TECHNOLOGY

Learning Outcomes: The student should have the ability to:
LO 9.1: Understand the fundamental concepts, architecture, and working principles of Adaptive Neuro-Fuzzy Inference Systems (ANFIS) and Neuro-Fuzzy Systems.
LO 9.2: Explain the integration of Artificial Neural Networks and Fuzzy Logic for developing adaptive intelligent systems.
LO 9.3: Analyze the five-layer architecture of ANFIS, including fuzzification, rule evaluation, normalization, consequent calculation, and output aggregation.
LO 9.4: Understand the design, training, and evaluation of Neuro-Fuzzy Systems for nonlinear modeling, prediction, and intelligent decision-making applications.

Course Outcome: Analyze and design Neuro-Fuzzy Systems and Adaptive Neuro-Fuzzy Inference Systems (ANFIS) by integrating artificial neural networks with fuzzy logic to develop adaptive intelligent models for nonlinear prediction, decision-making, and control applications.

Conclusion:

THAKUR ZCT'S
TCET
THAKUR COLLEGE OF ENGINEERING & TECHNOLOGY

For Faculty Use: -

Correction Parameters	Formative Assessment [40%]	Timely completion of Practical [40%]	Attendance / Learning Attitude [20%]	
Marks Obtained				